

STAR RX72N - rx_uart_comm Library
1.0.0

Generated by Doxygen 1.16.1

1 Directory Hierarchy	1
1.1 Directories	1
2 File Index	3
2.1 File List	3
3 Directory Documentation	5
3.1 libs/rx_uart_comm/inc Directory Reference	5
3.2 libs Directory Reference	5
3.3 libs/rx_uart_comm Directory Reference	6
3.4 libs/rx_uart_comm/src Directory Reference	6
4 File Documentation	7
4.1 libs/rx_uart_comm/inc/rx_uart_comm.h File Reference	7
4.1.1 Detailed Description	8
4.1.2 Data Structure Documentation	11
4.1.2.1 struct rx_uart_comm_handle_t	11
4.1.2.2 struct rx_uart_comm_config_t	15
4.1.3 Enumeration Type Documentation	16
4.1.3.1 rx_uart_comm_constants_t	16
4.1.4 Function Documentation	18
4.1.4.1 rx_uart_comm_deinit()	18
4.1.4.2 rx_uart_comm_init()	19
4.1.4.3 rx_uart_comm_receive()	21
4.1.4.4 rx_uart_comm_send()	23
4.2 rx_uart_comm.h	26
4.3 libs/rx_uart_comm/src/rx_uart_comm.c File Reference	33
4.3.1 Detailed Description	35
4.3.2 Enumeration Type Documentation	37
4.3.2.1 rx_receive_result_t	37
4.3.2.2 rx_uart_comm_header_size_t	37
4.3.2.3 rx_uart_comm_sequence_constants_t	37
4.3.2.4 rx_uart_comm_sync_constants_t	38
4.3.2.5 rx_uart_comm_sync_result_t	38
4.3.2.6 uart_crc32_seed_t	39
4.3.2.7 uart_dispatch_result_t	39
4.3.2.8 uart_frame_header_offset_t	39
4.3.3 Function Documentation	40
4.3.3.1 internal_align_to_sync()	40
4.3.3.2 internal_build_frame()	40
4.3.3.3 internal_compact_rx_buffer()	41
4.3.3.4 internal_decode_frame()	43
4.3.3.5 internal_decode_header()	44
4.3.3.6 internal_dispatch_frame()	45

4.3.3.7	<code>internal_find_sync()</code>	46
4.3.3.8	<code>internal_handle_no_sync()</code>	48
4.3.3.9	<code>internal_handle_ping()</code>	49
4.3.3.10	<code>internal_handle_reset()</code>	50
4.3.3.11	<code>internal_parse_header()</code>	51
4.3.3.12	<code>internal_read_uart_data()</code>	52
4.3.3.13	<code>internal_receive_iteration()</code>	53
4.3.3.14	<code>internal_verify_crc()</code>	55
4.3.3.15	<code>rx_uart_comm_deinit()</code>	56
4.3.3.16	<code>rx_uart_comm_init()</code>	57
4.3.3.17	<code>rx_uart_comm_receive()</code>	59
4.3.3.18	<code>rx_uart_comm_send()</code>	62
4.3.4	Variable Documentation	64
4.3.4.1	<code>s_tag</code>	64
4.3.4.2	<code>s_zero_handle</code>	65
4.4	<code>rx_uart_comm.c</code>	65

Chapter 1

Directory Hierarchy

1.1 Directories

inc	5
rx_uart_comm.h	7
libs	5
rx_uart_comm	6
inc	5
rx_uart_comm.h	7
src	6
rx_uart_comm.c	33
rx_uart_comm	6
inc	5
rx_uart_comm.h	7
src	6
rx_uart_comm.c	33
src	6
rx_uart_comm.c	33

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

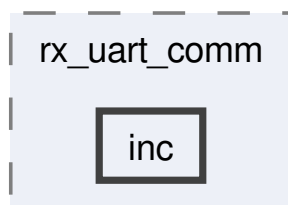
libs/rx_uart_comm/inc/ rx_uart_comm.h	
High-Level UART (SCI9) Communication Layer for RX72N	7
libs/rx_uart_comm/src/ rx_uart_comm.c	
High-Level UART (SCI9) Communication Layer Implementation	33

Chapter 3

Directory Documentation

3.1 libs/rx_uart_comm/inc Directory Reference

Directory dependency graph for inc:

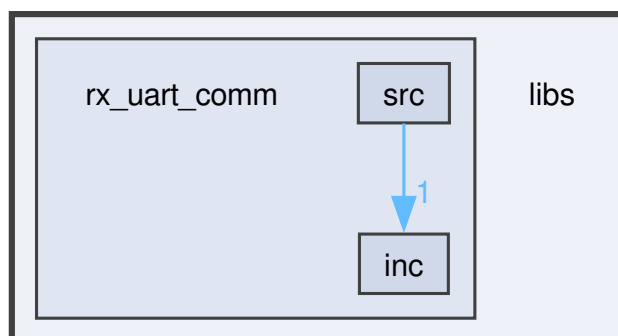


Files

- file [rx_uart_comm.h](#)
High-Level UART (SCI9) Communication Layer for RX72N.

3.2 libs Directory Reference

Directory dependency graph for libs:

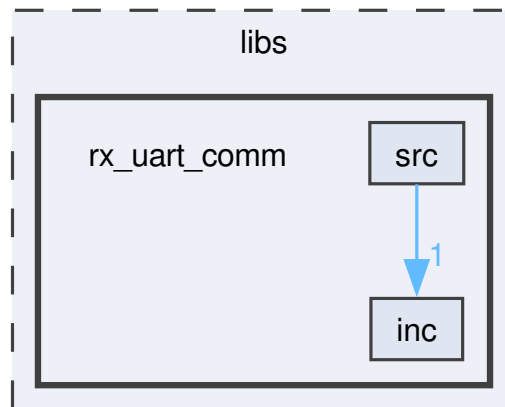


Directories

- directory [rx_uart_comm](#)

3.3 libs/rx_uart_comm Directory Reference

Directory dependency graph for rx_uart_comm:

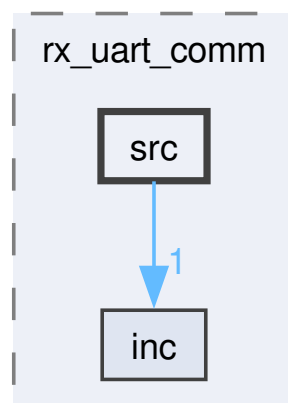


Directories

- directory [inc](#)
- directory [src](#)

3.4 libs/rx_uart_comm/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_uart_comm.c](#)
High-Level UART (SCI9) Communication Layer Implementation.

Chapter 4

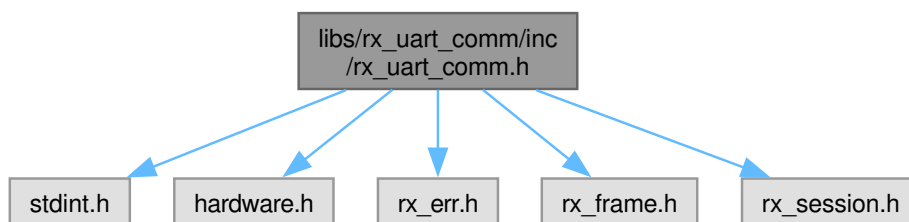
File Documentation

4.1 libs/rx_uart_comm/inc/rx_uart_comm.h File Reference

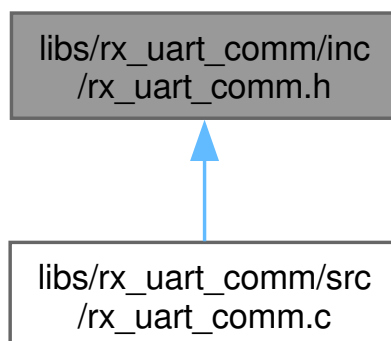
High-Level UART (SCI9) Communication Layer for RX72N.

```
#include <stdint.h>
#include "hardware.h"
#include "rx_err.h"
#include "rx_frame.h"
#include "rx_session.h"
```

Include dependency graph for rx_uart_comm.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_uart_comm_handle_t](#)
UART communication handle containing all operational state.
- struct [rx_uart_comm_config_t](#)
UART communication configuration options.

Enumerations

- enum [rx_uart_comm_constants_t](#) : `uint16_t` { `k_uart_comm_rx_buffer_size` = 2048 , `k_uart_comm_tx_buffer_size` = 2048 , `k_uart_comm_max_receive_iterations` = 1000 }
UART communication configuration constants.

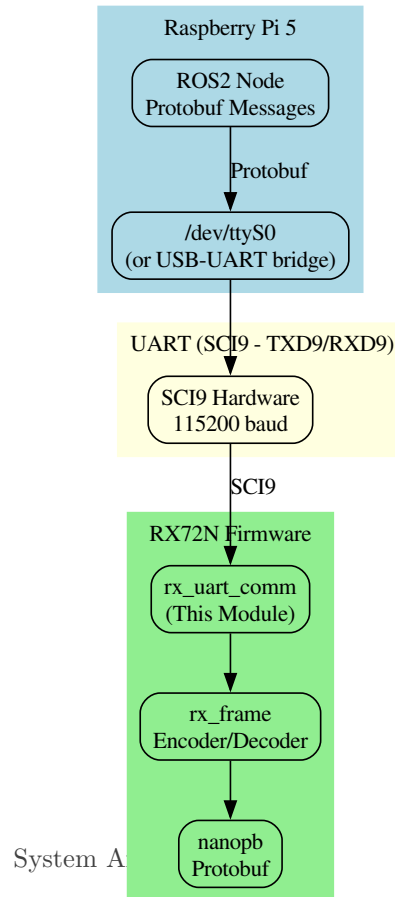
Functions

- `rx_err_t rx_uart_comm_init (rx_uart_comm_handle_t *handle, const rx_uart_comm_config_t *config)`
Initialize UART communication handle.
- `rx_err_t rx_uart_comm_deinit (rx_uart_comm_handle_t *handle)`
Deinitialize UART communication handle.
- `rx_err_t rx_uart_comm_send (rx_uart_comm_handle_t *handle, rx_frame_type_t type, uint8_t flags, const uint8_t *payload, uint32_t payload_len)`
Send a frame over UART (SCI9).
- `rx_err_t rx_uart_comm_receive (rx_uart_comm_handle_t *handle, rx_frame_t *frame, uint32_t timeout_ms)`
Receive and decode a frame from UART (SCI9).

4.1.1 Detailed Description

High-Level UART (SCI9) Communication Layer for RX72N.

Integrates the Frame Layer and SCI9 UART for reliable communication with the RPi5 controller. Provides the same frame-based protocol as USB and SPI but over a dedicated UART serial link. This module enables an alternative communication channel when USB CDC is unavailable or when a direct UART connection is preferred.



Features

- Frame encoding/decoding with CRC-32 error detection
- Automatic sequence number tracking for reliable delivery
- Non-blocking send/receive operations
- Same wire protocol as USB/SPI (interchangeable transport)
- Configurable UART channel (default: `k_uart_channel_9` for SCI9)

Frame Protocol

Uses the same frame format as USB and SPI communication:

```

+-----+-----+-----+-----+-----+-----+-----+
| SYNC | TYPE | FLAG | SEQ  | LENGTH | PAYLOAD | CRC32 |
| 0xAA | 1B  | 1B  | 2B  | 2B    | N bytes | 4B    |
+-----+-----+-----+-----+-----+-----+

```

UART Channel Assignment

Channel	SCI Channel	Baud Rate	Purpose
SCI9	<code>k_uart_channel_9</code>	115200	Binary protocol (nanopb)

Debug Output Conflict

Warning

When UART comm is active on SCI9, debug log output to SCI9 is suppressed. Callers should redirect debug output to USB CDC Port 1 for development and debugging. This is a known conflict documented in issue #418.

Performance Characteristics

Metric	Value	Notes
Baud rate	115200 bps	Configurable via <code>uart_init</code>
Frame latency	~1-10 ms	Depends on UART baud rate
Max frame size	2048 bytes	Buffer size limit

Memory Usage

Component	Size	Description
rx_uart_comm_handle_t	~4.2 KB	Per-instance handle
RX buffer	2048 B	Staging buffer for receives
TX buffer	2048 B	Staging buffer for sends
Frame encoder/decoder	~100 B	Internal state machines

Thread Safety

The handle is NOT thread-safe. External synchronization (ThreadX mutex) is required for concurrent access from multiple threads.

Module Dependencies

- `rx_frame.h`: Frame encoding/decoding layer
- `hardware.h`: UART HAL (`uart_write_channel`, `uart_read_channel`, `uart_rx_available`)
- `rx_err.h`: Error code definitions
- `rx_session.h`: Cross-transport sequence continuity

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp, or recursion
- Rule 2: [OK] Loop bounds via `k_uart_comm_max_receive_iterations`
- Rule 3: [OK] No dynamic memory (static buffers in handle)
- Rule 4: [OK] Functions are concise
- Rule 5: [OK] Input validation on all public functions
- Rule 6: [OK] Variables declared at smallest scope
- Rule 7: [OK] All return values checked
- Rule 8: [OK] Constants use C23 typed enums
- Rule 10: [OK] Compiles with `-Wall -Wextra -Werror`

SOLID Principles

- S (SRP): UART communication only, frame details in `rx_frame`
- O (OCP): Configurable channel via [rx_uart_comm_config_t](#)
- L (LSP): Same protocol as `rx_usb_comm` / `rx_spi_comm` (interchangeable)
- I (ISP): Clean send/receive interface
- D (DIP): Depends on `hardware.h` HAL abstraction

See also

`rx_frame.h` Frame layer implementation

`hardware.h` UART HAL

`rx_usb_comm.h` USB CDC communication (same protocol)

`rx_spi_comm.h` SPI communication (same protocol)

`docs/sections/01_nanopb_protocol.tex` Protocol specification

Author

Locked, Inc.

Date

2026-03-11

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Since

Version 1.0.0

Definition in file [rx_uart_comm.h](#).

4.1.2 Data Structure Documentation

4.1.2.1 struct rx_uart_comm_handle_t

UART communication handle containing all operational state.

Contains all state for UART communication including frame encoder/decoder, sequence tracking, and staging buffers. One handle corresponds to one logical UART communication channel (SCI9).

Memory Layout

Offset	Size	Field	Description
0	~50 B	encoder	Frame encoder state machine
50	~50 B	decoder	Frame decoder state machine
100	2048 B	rx_buffer	RX staging buffer
2148	2048 B	tx_buffer	TX staging buffer
4196	4 B	rx_buffer_len	Valid bytes in RX buffer
4200	4 B	rx_buffer_pos	Current read position
4204	4-8 B	session	Shared session state pointer
4212	1 B	initialized	Init flag
4213	1 B	channel	UART channel (SCI channel num)
Total	**~4.2 KB**		

Thread Safety

This handle is NOT thread-safe. External synchronization via ThreadX mutex is required for concurrent access from multiple threads.

Usage Example

```
static rx_uart_comm_handle_t s_uart_comm;

void uart_comm_setup(void) {
    rx_uart_comm_config_t config = {
        .session = &g_session,
        .channel = k_uart_channel_9
    };
    rx_err_t err = rx_uart_comm_init(&s_uart_comm, &config);
    if (err != k_rx_ok) {
        rx_log_error("UART_COMM", "Init failed");
        return;
    }
}
```

Invariant

```
initialized == 0 or initialized == 1
rx_buffer_pos <= rx_buffer_len
rx_buffer_len <= k_uart_comm_rx_buffer_size
```

Warning

Do not access fields directly; use API functions

Attention

Handle must be initialized via `rx_uart_comm_init()` before use

See also

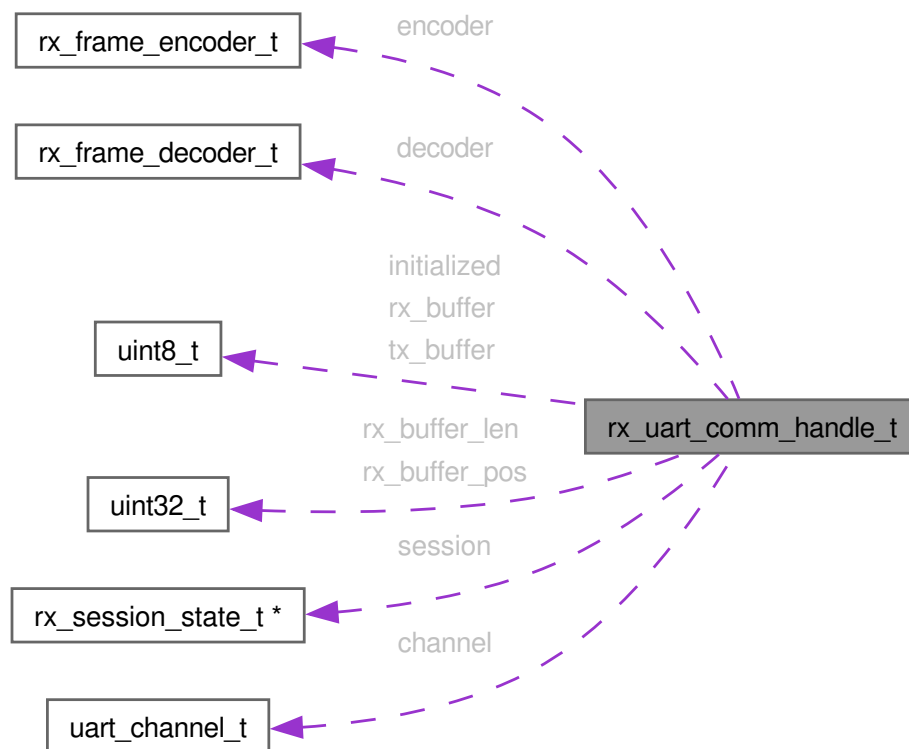
[rx_uart_comm_init\(\)](#) Initialize handle
[rx_uart_comm_deinit\(\)](#) Cleanup handle
[rx_uart_comm_config_t](#) Configuration options

Since

Version 1.0.0

Definition at line 262 of file [rx_uart_comm.h](#).

Collaboration diagram for rx_uart_comm_handle_t:



Data Fields

<code>uart_channel_t</code>	<code>channel</code>	UART SCI channel used by this handle. Stores the <code>uart_channel_t</code> value configured during init. Default is <code>k_uart_channel_9</code> (SCI9). See also <code>uart_channel_t</code>
-----------------------------	----------------------	--

<code>rx_frame_decoder_t</code>	decoder	Frame decoder state machine. Decodes incoming bytes, validates CRC, extracts payload. See also <code>rx_frame_decoder_t</code>
<code>rx_frame_encoder_t</code>	encoder	Frame encoder state machine. Encodes payloads into framed format with sync, header, and CRC. See also <code>rx_frame_encoder_t</code>
<code>uint8_t</code>	initialized	Initialization status flag. Set to 1 after successful <code>rx_uart_comm_init()</code>, 0 otherwise. Values: 0 = not initialized, 1 = initialized
<code>uint8_t</code>	<code>rx_buffer[k_uart_comm_rx_buffer_size]</code>	RX staging buffer. Accumulates incoming UART data before frame decoding. Size: <code>k_uart_comm_rx_buffer_size</code> (2048 bytes) Invariant Valid data in <code>[0, rx_buffer_len)</code>
<code>uint32_t</code>	<code>rx_buffer_len</code>	Number of valid bytes in RX buffer. Bytes <code>[0, rx_buffer_len)</code> contain unprocessed data. Invariant <code>rx_buffer_len <= k_uart_comm_rx_buffer_size</code>
<code>uint32_t</code>	<code>rx_buffer_pos</code>	Current read position in RX buffer. Next byte to process is <code>rx_buffer[rx_buffer_pos]</code>. Invariant <code>rx_buffer_pos <= rx_buffer_len</code>

<code>rx_session_state_t *</code>	<code>session</code>	Shared session state for cross-transport sequence continuity. Points to a single <code>rx_session_state_t</code> instance owned by the <code>comm_manager</code>. All transports share this state so that sequence numbers remain continuous across transport switches. See also <code>rx_session_state_t</code>
<code>uint8_t</code>	<code>tx_buffer[k_uart_comm_tx_buffer_size]</code>	TX staging buffer. Holds assembled frame for UART transmission. Size: <code>k_uart_comm_tx_buffer_size</code> (2048 bytes)

4.1.2.2 struct rx_uart_comm_config_t

UART communication configuration options.

Configuration passed to `rx_uart_comm_init()`. The session pointer is required for sequence tracking; the channel selects the SCI peripheral.

Default Values

Field	Default	Description
<code>session</code>	(required)	Shared session state pointer
<code>channel</code>	<code>k_uart_channel_9</code>	SCI9 is the default RPi5 UART port

Usage Example

```
static rx_session_state_t g_session;
rx_session_init(&g_session);

rx_uart_comm_config_t config = {
    .session = &g_session,
    .channel = k_uart_channel_9
};
rx_uart_comm_init(&handle, &config);
```

See also

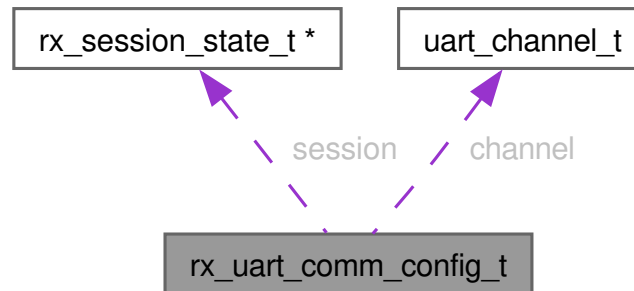
`rx_uart_comm_init()` Uses this configuration

Since

Version 1.0.0

Definition at line 368 of file [rx_uart_comm.h](#).

Collaboration diagram for rx_uart_comm_config_t:



Data Fields

uart_channel_t	channel	UART SCI channel to use. Selects which SCI channel to use for UART communication. Use <code>k_uart_channel_9</code> for SCI9 (the RPi5 UART interface). Default: <code>k_uart_channel_9</code> See also <code>uart_channel_t</code>
rx_session_state_t *	session	Shared session state for cross-transport sequence tracking. Pointer to the session state owned by <code>rx_comm_manager</code>. Must be initialized via <code>rx_session_init()</code> before passing to rx_uart_comm_init(). Required - must not be NULL. See also <code>rx_session_state_t</code>

4.1.3 Enumeration Type Documentation

4.1.3.1 rx_uart_comm_constants_t

enum [rx_uart_comm_constants_t](#) : uint16_t

UART communication configuration constants.

Compile-time constants for UART communication buffer sizes and loop bounds. These values are tuned for the RX72N SCI9 UART implementation with typical control message sizes.

Buffer Sizing Rationale

- 2048 bytes accommodates largest expected protobuf message + frame overhead
- Provides headroom for frame header (8 bytes) and CRC (4 bytes)

Memory Impact

Total per-handle: ~4.2 KB (2 x 2048 buffers + encoder/decoder state)

Invariant

Buffer sizes must be $\geq$ maximum frame size

k_uart_comm_max_receive_iterations bounds all receive loops

Since

Version 1.0.0

Enumerator

k_uart_comm_rx_buffer_size	RX staging buffer size in bytes. Buffer for accumulating incoming UART data before frame decoding. Must be large enough for largest expected frame + partial data. Value: 2048 bytes
k_uart_comm_tx_buffer_size	TX staging buffer size in bytes. Buffer for assembled frames before UART transmission. Must accommodate frame header + max payload + CRC. Value: 2048 bytes
k_uart_comm_max_receive_iterations	Maximum iterations for receive loop (NASA Rule 2). Provides static upper bound for receive polling loops to ensure bounded execution time even under pathological conditions. Value: 1000 iterations Warning Reaching this limit indicates UART stall or protocol error

Definition at line 170 of file [rx_uart_comm.h](#).

```
00170         : uint16_t {
00171     /**
00172     * @brief RX staging buffer size in bytes
00173     * @details
00174     * Buffer for accumulating incoming UART data before frame decoding.
```

```

00175  * Must be large enough for largest expected frame + partial data.
00176  * @par Value: 2048 bytes
00177  */
00178  k_uart_comm_rx_buffer_size = 2048,
00179
00180  /**
00181  * @brief TX staging buffer size in bytes
00182  * @details
00183  * Buffer for assembled frames before UART transmission.
00184  * Must accommodate frame header + max payload + CRC.
00185  * @par Value: 2048 bytes
00186  */
00187  k_uart_comm_tx_buffer_size = 2048,
00188
00189  /**
00190  * @brief Maximum iterations for receive loop (NASA Rule 2)
00191  * @details
00192  * Provides static upper bound for receive polling loops to ensure
00193  * bounded execution time even under pathological conditions.
00194  * @par Value: 1000 iterations
00195  * @warning Reaching this limit indicates UART stall or protocol error
00196  */
00197  k_uart_comm_max_receive_iterations = 1000,
00198 } rx_uart_comm_constants_t;

```

4.1.4 Function Documentation

4.1.4.1 rx_uart_comm_deinit()

```

rx_err_t rx_uart_comm_deinit (
    rx_uart_comm_handle_t * handle) [nodiscard]

```

Deinitialize UART communication handle.

Clears the initialized flag to prevent further use of the handle. Does not deinitialize the underlying UART hardware.

Parameters

in,out	handle	Pointer to initialized handle
--------	--------	-------------------------------

Returns

k_rx_ok on success, k_rx_err_invalid_arg if handle is nullptr

Precondition

handle must not be nullptr

handle must have been initialized via [rx_uart_comm_init\(\)](#)

Postcondition

handle->initialized == 0

Further calls to rx_uart_comm_send/receive will return k_rx_err_invalid_state

Since

Version 1.0.0

Definition at line 711 of file [rx_uart_comm.c](#).

```

00712 {
00713     if (handle == nullptr) {
00714         return k_rx_err_invalid_arg;
00715     }
00716     if (handle->initialized) {
00717         (void)(rx_frame_encoder_deinit(&handle->encoder));
00718         /* Post-condition: deinit only fails on nullptr; &handle->encoder is never null */
00719         (void)(rx_frame_decoder_deinit(&handle->decoder));
00720         /* Post-condition: deinit only fails on nullptr; &handle->decoder is never null */
00721         handle->initialized = 0U;
00722     }
00723     rx_log_debug(s_tag, "UART comm deinitialized");
00724     return k_rx_ok;
00725 }

```

References [rx_uart_comm_handle_t::decoder](#), [rx_uart_comm_handle_t::encoder](#), [rx_uart_comm_handle_t::initialized](#) and [s_tag](#).

4.1.4.2 rx_uart_comm_init()

```

rx_err_t rx_uart_comm_init (
    rx_uart_comm_handle_t * handle,
    const rx_uart_comm_config_t * config) [nodiscard]

```

Initialize UART communication handle.

Sets up frame encoder/decoder, clears internal buffers, and prepares the handle for communication. The UART driver must be initialized separately before this handle can successfully send/receive data.

Algorithm

1. Validate handle pointer (NULL check)
2. Clear handle memory
3. Initialize frame encoder and decoder
4. Copy configuration (channel, session)
5. Set initialized flag

Parameters

out	handle	Pointer to handle to initialize • Must be valid non-nullptr • All fields will be overwritten • Caller maintains ownership
-----	--------	---

in	config	Configuration options (must not be NULL) • session: Required shared session state pointer • channel: UART channel (<code>k_uart_channel_9</code> for SCI9) • Structure is copied, need not persist after call
----	--------	---

Returns

`rx_err_t` Error code

Return values

<code>k_rx_ok</code>	Handle initialized successfully
<code>k_rx_err_invalid_arg</code>	handle or config is nullptr

Precondition

handle points to allocated `rx_uart_comm_handle_t`
 config->session points to initialized `rx_session_state_t`

Postcondition

handle->initialized == 1
 Buffers cleared to zero

Note

Safe to call multiple times (reinitializes handle)
 Not thread-safe; call from single initialization context

See also

`rx_uart_comm_deinit()` Cleanup resources
`uart_channel_t` UART channel enumeration

Since

Version 1.0.0

Definition at line 683 of file `rx_uart_comm.c`.

```

00684 {
00685     if (handle == nullptr) {
00686         return k_rx_err_invalid_arg;
00687     }
00688     *handle = s_zero_handle;
00689
00690
00691     if (config == nullptr || config->session == nullptr) {
00692         return k_rx_err_invalid_arg;
00693     }

```

```

00694  handle->session = config->session;
00695  handle->channel = config->channel;
00696
00697  (void)(rx_frame_encoder_init(&handle->encoder));
00698  /* Post-condition: encoder init only fails on nullptr; &handle->encoder is never null */
00699
00700  (void)(rx_frame_decoder_init(&handle->decoder));
00701  /* Post-condition: decoder init only fails on nullptr; &handle->decoder is never null */
00702
00703  handle->rx_buffer_len = 0;
00704  handle->rx_buffer_pos = 0;
00705  handle->initialized = 1U;
00706
00707  rx_log_debug(s_tag, "UART comm initialized");
00708  return k_rx_ok;
00709 }

```

References [rx_uart_comm_config_t::channel](#), [rx_uart_comm_handle_t::channel](#), [rx_uart_comm_handle_t::decoder](#), [rx_uart_comm_handle_t::encoder](#), [rx_uart_comm_handle_t::initialized](#), [rx_uart_comm_handle_t::rx_buffer_len](#), [rx_uart_comm_handle_t::rx_buffer_pos](#), [s_tag](#), [s_zero_handle](#), [rx_uart_comm_config_t::session](#), and [rx_uart_comm_handle_t::session](#).

4.1.4.3 rx_uart_comm_receive()

```

rx_err_t rx_uart_comm_receive (
    rx_uart_comm_handle_t * handle,
    rx_frame_t * frame,
    uint32_t timeout_ms) [nodiscard]

```

Receive and decode a frame from UART (SCI9).

Reads available bytes from the UART RX buffer, feeds them through the frame decoder, validates CRC-32, and extracts the payload. Operates in polling mode (non-blocking); returns `k_rx_err_no_data` if no complete frame is available yet.

Algorithm

1. Validate handle and frame pointers
2. Check initialization
3. Check `uart_rx_available()` for pending data
4. If data available: read bytes via `uart_read_channel()`
5. Feed bytes to frame decoder
6. If frame complete: validate CRC, populate output frame
7. Update session RX sequence

Parameters

in,out	handle	Initialized handle • Must be initialized via rx_uart_comm_init() • Internal buffers and decoder state modified on each call
out	frame	Pointer to frame structure for output • Must be valid non-nullptr • All fields populated on success

in	timeout_ms	Timeout in milliseconds • 0: Poll once, return immediately if no complete frame • (timeout not currently used; always polls once)
----	------------	--

Returns

rx_err_t Error code

Return values

k_rx_ok	Complete frame received and decoded
k_rx_err_invalid_arg	handle or frame is nullptr
k_rx_err_invalid_state	Not initialized
k_rx_err_no_data	No complete frame available yet
k_rx_err_crc_mismatch	CRC-32 validation failed
k_rx_err_protocol_error	Invalid frame format

Precondition

handle initialized via [rx_uart_comm_init\(\)](#)
frame points to valid rx_frame_t

Postcondition

On success: frame contains decoded data
On success: session RX sequence updated

Note

Loop iterations bounded by k_uart_comm_max_receive_iterations
Thread safety: caller must provide external synchronization

See also

[rx_uart_comm_send\(\)](#) Send frames to peer

Since

Version 1.0.0

Definition at line 980 of file [rx_uart_comm.c](#).

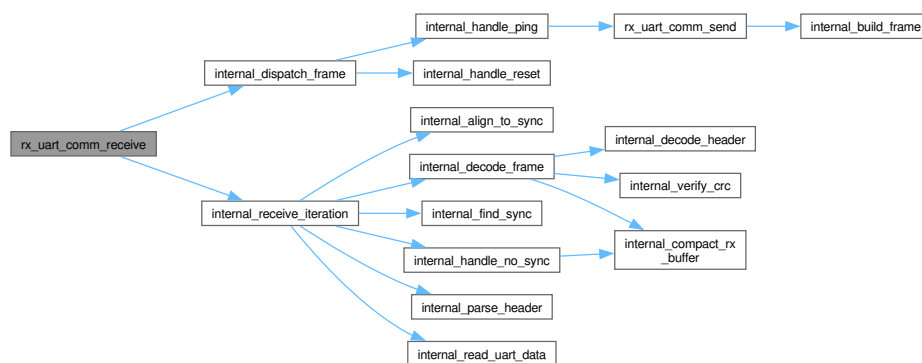
```

00981 {
00982   (void)timeout_ms; /* UART receive is non-blocking; timeout reserved for future use */
00983
00984   if (handle == nullptr) {
00985     return k_rx_err_invalid_arg;
00986   }
00987
00988   if (frame == nullptr) {
00989     return k_rx_err_invalid_arg;
00990   }
00991
00992   if (!handle->initialized) {
00993     return k_rx_err_invalid_state;
00994   }
00995
00996   rx_err_t err = k_rx_ok;
00997
00998   for (uint32_t iterations = 0; iterations < k_uart_comm_max_receive_iterations; iterations++) {
00999     rx_receive_result_t result = internal_receive_iteration(handle, frame, &err);
01000     if (result == k_receive_error) {
01001       return err;
01002     }
01003     if (result != k_receive_done) {
01004       continue;
01005     }
01006
01007     uart_dispatch_result_t dispatch = internal_dispatch_frame(handle, frame, &err);
01008     if (dispatch == k_dispatch_error) {
01009       return err;
01010     }
01011     if (dispatch == k_dispatch_done) {
01012       return k_rx_ok;
01013     }
01014     /* k_dispatch_continue: loop for next frame */
01015   }
01016
01017   return k_rx_err_timeout;
01018 }

```

References [rx_uart_comm_handle_t::initialized](#), [internal_dispatch_frame\(\)](#), [internal_receive_iteration\(\)](#), [k_dispatch_done](#), [k_dispatch_error](#), [k_receive_done](#), [k_receive_error](#), and [k_uart_comm_max_receive_iterations](#).

Here is the call graph for this function:



4.1.4.4 rx_uart_comm_send()

```

rx_err_t rx_uart_comm_send (
    rx_uart_comm_handle_t * handle,
    rx_frame_type_t type,
    uint8_t flags,

```

```
const uint8_t * payload,
uint32_t payload_len)  [nodiscard]
```

Send a frame over UART (SCI9).

Encodes the payload into a frame with automatic sequence number assignment and CRC-32 calculation, then transmits it via `uart_write_channel()`. This operation is non-blocking once the data is queued in the UART FIFO.

Algorithm

1. Validate handle and payload pointers
2. Check initialization
3. Check payload fits in TX buffer
4. Encode frame with header, payload, and CRC-32
5. Write encoded frame to UART via `uart_write_channel()`
6. Increment session TX sequence counter

Parameters

in,out	handle	Initialized handle • Must be initialized via <code>rx_uart_comm_init()</code> • Session TX sequence incremented on success
in	type	Frame type (typically <code>k_frame_type_response</code>) • See <code>rx_frame_type_t</code> for valid types
in	flags	Frame flags (ORed together) • See <code>rx_frame_flags_t</code> for valid flags; 0 for no flags
in	payload	Pointer to payload data • Must not be nullptr (even for zero-length payload) • Copied to internal buffer (no ownership transfer)
in	payload_len	Payload length in bytes • Max: <code>k_uart_comm_tx_buffer_size</code> - frame overhead • 0 is valid (empty payload)

Returns

`rx_err_t` Error code

Return values

<code>k_rx_ok</code>	Frame sent successfully
----------------------	-------------------------

<code>k_rx_err_invalid_arg</code>	handle or payload is nullptr
<code>k_rx_err_invalid_state</code>	Not initialized
<code>k_rx_err_invalid_size</code>	Payload exceeds maximum frame size
<code>k_rx_err_hw_error</code>	UART hardware write failure (propagated from <code>uart_write_channel()</code>)
other	Propagated from <code>rx_session_next_tx()</code> or <code>rx_frame_encode()</code>

Precondition

handle initialized via `rx_uart_comm_init()`
 payload != nullptr

Postcondition

On success: session TX sequence incremented
 On success: frame transmitted via UART
 On failure: no state changes

Note

Thread safety: caller must provide external synchronization

Warning

When SCI9 is used for comm, debug log output to SCI9 is suppressed

See also

`rx_uart_comm_receive()` Receive frames from peer

Since

Version 1.0.0

Definition at line 784 of file `rx_uart_comm.c`.

```

00789 {
00790     if (handle == nullptr) {
00791         return k_rx_err_invalid_arg;
00792     }
00793
00794     if (!handle->initialized) {
00795         rx_log_error(s_tag, "Handle not initialized");
00796         return k_rx_err_invalid_state;
00797     }
00798
00799     if (payload == nullptr && payload_len > 0) {
00800         return k_rx_err_invalid_arg;
00801     }
00802
00803     if (payload_len > k_frame_max_payload) {
00804         rx_log_error(s_tag, "Payload too large");
00805         return k_rx_err_invalid_size;
00806     }
00807
00808     uint16_t sequence = k_initial_sequence;
00809     rx_err_t err = rx_session_next_tx(handle->session, &sequence);
00810     if (err != k_rx_ok) {

```

```

00811     rx_log_error(s_tag, "Session next_tx failed");
00812     return err;
00813 }
00814
00815 rx_frame_t frame = {};
00816 /* build_frame only fails on nullptr frame (impossible: &frame), payload_len > max
00817 * (already validated above), or null payload with nonzero len (validated above) */
00818 (void)internal_build_frame(&frame, sequence, type, flags, payload, payload_len);
00819
00820 uint32_t wire_len = 0;
00821 err = rx_frame_encode(&handle->encoder, &frame, handle->tx_buffer, &wire_len);
00822 if (err != k_rx_ok) {
00823     rx_log_error(s_tag, "Frame encode failed");
00824     return err;
00825 }
00826
00827 err = uart_write_channel(handle->channel, handle->tx_buffer, (uint16_t)wire_len);
00828 if (err != k_rx_ok) {
00829     rx_log_error(s_tag, "UART write failed");
00830     return err;
00831 }
00832
00833 return k_rx_ok;
00834 }

```

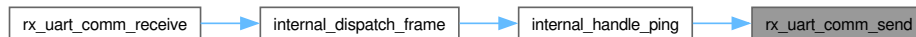
References [rx_uart_comm_handle_t::channel](#), [rx_uart_comm_handle_t::encoder](#), [rx_uart_comm_handle_t::initialize](#), [internal_build_frame\(\)](#), [k_initial_sequence](#), [s_tag](#), [rx_uart_comm_handle_t::session](#), and [rx_uart_comm_handle_t::](#)

Referenced by [internal_handle_ping\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.2 rx_uart_comm.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_uart_comm.h
00003  * @brief High-Level UART (SCI9) Communication Layer for RX72N
00004  *
00005  * @details
00006  * Integrates the Frame Layer and SCI9 UART for reliable communication with the
00007  * RPi5 controller. Provides the same frame-based protocol as USB and SPI but over
00008  * a dedicated UART serial link. This module enables an alternative communication
00009  * channel when USB CDC is unavailable or when a direct UART connection is preferred.
00010  *
00011  * @par System Architecture
00012  * @dot
00013  * digraph uart_comm_architecture {
00014  *     rankdir=TB;
00015  *     node [shape=box, style=rounded];
00016  *
00017  *     subgraph cluster_rpi5 {
00018  *         label="Raspberry Pi 5";
00019  *         style=filled;
00020  *         color=lightblue;
00021  *         ttyS [label="/dev/ttyS0\n(or USB-UART bridge)"];
00022  *         ros2 [label="ROS2 Node\nProtobuf Messages"];
00023  *     }
00024  *
00025  *     subgraph cluster_uart {
00026  *         label="UART (SCI9 - TXD9/RXD9)";
00027  *         style=filled;

```

```

00028 *   color=lightyellow;
00029 *   uart_hw [label="SCI9 Hardware\n115200 baud"];
00030 * }
00031 *
00032 * subgraph cluster_rx72n {
00033 *   label="RX72N Firmware";
00034 *   style=filled;
00035 *   color=lightgreen;
00036 *   uart_comm [label="rx_uart_comm\n(This Module)"];
00037 *   frame [label="rx_frame\nEncoder/Decoder"];
00038 *   nanopb [label="nanopb\nProtobuf"];
00039 * }
00040 *
00041 * ros2 -> ttyS [label="Protobuf"];
00042 * ttyS -> uart_hw;
00043 * uart_hw -> uart_comm [label="SCI9"];
00044 * uart_comm -> frame;
00045 * frame -> nanopb;
00046 * }
00047 * @enddot
00048 *
00049 * @par Features
00050 * - Frame encoding/decoding with CRC-32 error detection
00051 * - Automatic sequence number tracking for reliable delivery
00052 * - Non-blocking send/receive operations
00053 * - Same wire protocol as USB/SPI (interchangeable transport)
00054 * - Configurable UART channel (default: k_uart_channel_9 for SCI9)
00055 *
00056 * @par Frame Protocol
00057 * Uses the same frame format as USB and SPI communication:
00058 * ```
00059 * +-----+-----+-----+-----+-----+-----+
00060 * | SYNC | TYPE | FLAG | SEQ | LENGTH | PAYLOAD | CRC32 |
00061 * | 0xAA | 1B  | 1B  | 2B  | 2B   | N bytes | 4B   |
00062 * +-----+-----+-----+-----+-----+-----+
00063 * ```
00064 *
00065 * @par UART Channel Assignment
00066 * | Channel | SCI Channel | Baud Rate | Purpose |
00067 * |-----|-----|-----|-----|
00068 * | SCI9   | k_uart_channel_9 | 115200 | Binary protocol (nanopb) |
00069 *
00070 * @par Debug Output Conflict
00071 * @warning When UART comm is active on SCI9, debug log output to SCI9 is
00072 * suppressed. Callers should redirect debug output to USB CDC Port 1 for
00073 * development and debugging. This is a known conflict documented in issue #418.
00074 *
00075 * @par Performance Characteristics
00076 * | Metric | Value | Notes |
00077 * |-----|-----|-----|
00078 * | Baud rate | 115200 bps | Configurable via uart_init |
00079 * | Frame latency | ~1-10 ms | Depends on UART baud rate |
00080 * | Max frame size | 2048 bytes | Buffer size limit |
00081 *
00082 * @par Memory Usage
00083 * | Component | Size | Description |
00084 * |-----|-----|-----|
00085 * | rx_uart_comm_handle_t | ~4.2 KB | Per-instance handle |
00086 * | RX buffer | 2048 B | Staging buffer for receives |
00087 * | TX buffer | 2048 B | Staging buffer for sends |
00088 * | Frame encoder/decoder | ~100 B | Internal state machines |
00089 *
00090 * @par Thread Safety
00091 * The handle is **NOT thread-safe**. External synchronization (ThreadX mutex)
00092 * is required for concurrent access from multiple threads.
00093 *
00094 * @par Module Dependencies
00095 * - rx_frame.h: Frame encoding/decoding layer
00096 * - hardware.h: UART HAL (uart_write_channel, uart_read_channel, uart_rx_available)
00097 * - rx_err.h: Error code definitions
00098 * - rx_session.h: Cross-transport sequence continuity
00099 *
00100 * @par NASA Power of 10 Compliance
00101 * - Rule 1: [OK] No goto, setjmp, or recursion
00102 * - Rule 2: [OK] Loop bounds via k_uart_comm_max_receive_iterations
00103 * - Rule 3: [OK] No dynamic memory (static buffers in handle)
00104 * - Rule 4: [OK] Functions are concise
00105 * - Rule 5: [OK] Input validation on all public functions
00106 * - Rule 6: [OK] Variables declared at smallest scope
00107 * - Rule 7: [OK] All return values checked
00108 * - Rule 8: [OK] Constants use C23 typed enums
00109 * - Rule 10: [OK] Compiles with -Wall -Wextra -Werror
00110 *
00111 * @par SOLID Principles
00112 * - **S (SRP):** UART communication only, frame details in rx_frame
00113 * - **O (OCP):** Configurable channel via rx_uart_comm_config_t
00114 * - **L (LSP):** Same protocol as rx_usb_comm / rx_spi_comm (interchangeable)

```

```

00115 * - **I (ISP):** Clean send/receive interface
00116 * - **D (DIP):** Depends on hardware.h HAL abstraction
00117 *
00118 * @see rx_frame.h Frame layer implementation
00119 * @see hardware.h UART HAL
00120 * @see rx_usb_comm.h USB CDC communication (same protocol)
00121 * @see rx_spi_comm.h SPI communication (same protocol)
00122 * @see docs/sections/01_nanopb_protocol.tex Protocol specification
00123 *
00124 * @author Locked, Inc.
00125 * @date 2026-03-11
00126 * @copyright Copyright (c) 2026 Locked Inc.
00127 * SPDX-License-Identifier: MIT
00128 * @since Version 1.0.0
00129 */
00130
00131 #pragma once
00132
00133 #include <stdint.h>
00134
00135 #include "hardware.h"
00136 #include "rx_err.h"
00137 #include "rx_frame.h"
00138 #include "rx_session.h"
00139
00140 #ifdef __cplusplus
00141 extern "C" {
00142 #endif
00143
00144 /*
=====
00145 * Configuration Constants
00146 *
=====
00147 */
00148
00149 /**
00150 * @enum rx_uart_comm_constants_t
00151 * @brief UART communication configuration constants
00152 *
00153 * @details
00154 * Compile-time constants for UART communication buffer sizes and loop bounds.
00155 * These values are tuned for the RX72N SCI9 UART implementation with typical
00156 * control message sizes.
00157 *
00158 * @par Buffer Sizing Rationale
00159 * - 2048 bytes accommodates largest expected protobuf message + frame overhead
00160 * - Provides headroom for frame header (8 bytes) and CRC (4 bytes)
00161 *
00162 * @par Memory Impact
00163 * Total per-handle: ~4.2 KB (2 x 2048 buffers + encoder/decoder state)
00164 *
00165 * @invariant Buffer sizes must be >= maximum frame size
00166 * @invariant k_uart_comm_max_receive_iterations bounds all receive loops
00167 *
00168 * @since Version 1.0.0
00169 */
00170 typedef enum : uint16_t {
00171 /**
00172 * @brief RX staging buffer size in bytes
00173 * @details
00174 * Buffer for accumulating incoming UART data before frame decoding.
00175 * Must be large enough for largest expected frame + partial data.
00176 * @par Value: 2048 bytes
00177 */
00178 k_uart_comm_rx_buffer_size = 2048,
00179
00180 /**
00181 * @brief TX staging buffer size in bytes
00182 * @details
00183 * Buffer for assembled frames before UART transmission.
00184 * Must accommodate frame header + max payload + CRC.
00185 * @par Value: 2048 bytes
00186 */
00187 k_uart_comm_tx_buffer_size = 2048,
00188
00189 /**
00190 * @brief Maximum iterations for receive loop (NASA Rule 2)
00191 * @details
00192 * Provides static upper bound for receive polling loops to ensure
00193 * bounded execution time even under pathological conditions.
00194 * @par Value: 1000 iterations
00195 * @warning Reaching this limit indicates UART stall or protocol error
00196 */
00197 k_uart_comm_max_receive_iterations = 1000,
00198 } rx_uart_comm_constants_t;
00199

```

```

00200 /*
=====
00201 * Handle and Configuration
00202 *
=====
00203 */
00204
00205 /**
00206  * @struct rx_uart_comm_handle_t
00207  * @brief UART communication handle containing all operational state
00208
00209  * @details
00210  * Contains all state for UART communication including frame encoder/decoder,
00211  * sequence tracking, and staging buffers. One handle corresponds to one
00212  * logical UART communication channel (SCI9).
00213  *
00214  * @par Memory Layout
00215  * | Offset | Size | Field | Description |
00216  * |-----|-----|-----|-----|
00217  * | 0 | ~50 B | encoder | Frame encoder state machine |
00218  * | 50 | ~50 B | decoder | Frame decoder state machine |
00219  * | 100 | 2048 B | rx_buffer | RX staging buffer |
00220  * | 2148 | 2048 B | tx_buffer | TX staging buffer |
00221  * | 4196 | 4 B | rx_buffer_len | Valid bytes in RX buffer |
00222  * | 4200 | 4 B | rx_buffer_pos | Current read position |
00223  * | 4204 | 4-8 B | session | Shared session state pointer |
00224  * | 4212 | 1 B | initialized | Init flag |
00225  * | 4213 | 1 B | channel | UART channel (SCI channel num) |
00226  * | **Total** | **~4.2 KB** | | |
00227  *
00228  * @par Thread Safety
00229  * This handle is **NOT thread-safe**. External synchronization via ThreadX
00230  * mutex is required for concurrent access from multiple threads.
00231  *
00232  * @par Usage Example
00233  * @code
00234  * static rx_uart_comm_handle_t s_uart_comm;
00235  *
00236  * void uart_comm_setup(void) {
00237  *     rx_uart_comm_config_t config = {
00238  *         .session = &g_session,
00239  *         .channel = k_uart_channel_9
00240  *     };
00241  *     rx_err_t err = rx_uart_comm_init(&s_uart_comm, &config);
00242  *     if (err != k_rx_ok) {
00243  *         rx_log_error("UART_COMM", "Init failed");
00244  *         return;
00245  *     }
00246  * }
00247  * @endcode
00248  *
00249  * @invariant initialized == 0 or initialized == 1
00250  * @invariant rx_buffer_pos <= rx_buffer_len
00251  * @invariant rx_buffer_len <= k_uart_comm_rx_buffer_size
00252  *
00253  * @warning Do not access fields directly; use API functions
00254  * @attention Handle must be initialized via rx_uart_comm_init() before use
00255  *
00256  * @see rx_uart_comm_init() Initialize handle
00257  * @see rx_uart_comm_deinit() Cleanup handle
00258  * @see rx_uart_comm_config_t Configuration options
00259  *
00260  * @since Version 1.0.0
00261  */
00262 typedef struct {
00263     /**
00264      * @brief Frame encoder state machine
00265      * @details Encodes payloads into framed format with sync, header, and CRC.
00266      * @see rx_frame_encoder_t
00267      */
00268     rx_frame_encoder_t encoder;
00269
00270     /**
00271      * @brief Frame decoder state machine
00272      * @details Decodes incoming bytes, validates CRC, extracts payload.
00273      * @see rx_frame_decoder_t
00274      */
00275     rx_frame_decoder_t decoder;
00276
00277     /**
00278      * @brief RX staging buffer
00279      * @details
00280      * Accumulates incoming UART data before frame decoding.
00281      * @par Size: k_uart_comm_rx_buffer_size (2048 bytes)
00282      * @invariant Valid data in [0, rx_buffer_len)
00283      */
00284     uint8_t rx_buffer[k_uart_comm_rx_buffer_size];

```

```

00285
00286 /**
00287  * @brief TX staging buffer
00288  * @details
00289  * Holds assembled frame for UART transmission.
00290  * @par Size: k_uart_comm_tx_buffer_size (2048 bytes)
00291  */
00292 uint8_t tx_buffer[k_uart_comm_tx_buffer_size];
00293
00294 /**
00295  * @brief Number of valid bytes in RX buffer
00296  * @details
00297  * Bytes [0, rx_buffer_len) contain unprocessed data.
00298  * @invariant rx_buffer_len <= k_uart_comm_rx_buffer_size
00299  */
00300 uint32_t rx_buffer_len;
00301
00302 /**
00303  * @brief Current read position in RX buffer
00304  * @details
00305  * Next byte to process is rx_buffer[rx_buffer_pos].
00306  * @invariant rx_buffer_pos <= rx_buffer_len
00307  */
00308 uint32_t rx_buffer_pos;
00309
00310 /**
00311  * @brief Shared session state for cross-transport sequence continuity
00312  * @details
00313  * Points to a single rx_session_state_t instance owned by the comm_manager.
00314  * All transports share this state so that sequence numbers remain continuous
00315  * across transport switches.
00316  * @see rx_session_state_t
00317  */
00318 rx_session_state_t* session;
00319
00320 /**
00321  * @brief Initialization status flag
00322  * @details
00323  * Set to 1 after successful rx_uart_comm_init(), 0 otherwise.
00324  * @par Values: 0 = not initialized, 1 = initialized
00325  */
00326 uint8_t initialized;
00327
00328 /**
00329  * @brief UART SCI channel used by this handle
00330  * @details
00331  * Stores the uart_channel_t value configured during init.
00332  * Default is k_uart_channel_9 (SCI9).
00333  * @see uart_channel_t
00334  */
00335 uart_channel_t channel;
00336 } rx_uart_comm_handle_t;
00337
00338 /**
00339  * @struct rx_uart_comm_config_t
00340  * @brief UART communication configuration options
00341  * @details
00342  * Configuration passed to rx_uart_comm_init(). The session pointer is
00343  * required for sequence tracking; the channel selects the SCI peripheral.
00344  *
00345  * @par Default Values
00346  * | Field | Default | Description |
00347  * |-----|-----|-----|
00348  * | session | (required) | Shared session state pointer |
00349  * | channel | k_uart_channel_9 | SCI9 is the default RPi5 UART port |
00350  *
00351  * @par Usage Example
00352  * @code
00353  * static rx_session_state_t g_session;
00354  * rx_session_init(&g_session);
00355  *
00356  * rx_uart_comm_config_t config = {
00357  *     .session = &g_session,
00358  *     .channel = k_uart_channel_9
00359  * };
00360  * rx_uart_comm_init(&handle, &config);
00361  * @endcode
00362  *
00363  * @see rx_uart_comm_init() Uses this configuration
00364  *
00365  * @since Version 1.0.0
00366  */
00367
00368 typedef struct {
00369     /**
00370      * @brief Shared session state for cross-transport sequence tracking
00371      * @details

```

```

00372 * Pointer to the session state owned by rx_comm_manager. Must be
00373 * initialized via rx_session_init() before passing to rx_uart_comm_init().
00374 * Required - must not be NULL.
00375 * @see rx_session_state_t
00376 */
00377 rx_session_state_t* session;
00378
00379 /**
00380 * @brief UART SCI channel to use
00381 * @details
00382 * Selects which SCI channel to use for UART communication.
00383 * Use k_uart_channel_9 for SCI9 (the RPi5 UART interface).
00384 * @par Default: k_uart_channel_9
00385 * @see uart_channel_t
00386 */
00387 uart_channel_t channel;
00388 } rx_uart_comm_config_t;
00389
00390 /*
=====
00391 * Initialization
00392 *
=====
00393 */
00394
00395 /**
00396 * @brief Initialize UART communication handle
00397 *
00398 * @details
00399 * Sets up frame encoder/decoder, clears internal buffers, and prepares the
00400 * handle for communication. The UART driver must be initialized separately
00401 * before this handle can successfully send/receive data.
00402 *
00403 * @par Algorithm
00404 * 1. Validate handle pointer (NULL check)
00405 * 2. Clear handle memory
00406 * 3. Initialize frame encoder and decoder
00407 * 4. Copy configuration (channel, session)
00408 * 5. Set initialized flag
00409 *
00410 * @param[out] handle Pointer to handle to initialize
00411 * - Must be valid non-nullptr
00412 * - All fields will be overwritten
00413 * - Caller maintains ownership
00414 * @param[in] config Configuration options (must not be NULL)
00415 * - session: Required shared session state pointer
00416 * - channel: UART channel (k_uart_channel_9 for SCI9)
00417 * - Structure is copied, need not persist after call
00418 *
00419 * @return rx_err_t Error code
00420 * @retval k_rx_ok Handle initialized successfully
00421 * @retval k_rx_err_invalid_arg handle or config is nullptr
00422 *
00423 * @pre handle points to allocated rx_uart_comm_handle_t
00424 * @pre config->session points to initialized rx_session_state_t
00425 * @post handle->initialized == 1
00426 * @post Buffers cleared to zero
00427 *
00428 * @note Safe to call multiple times (reinitializes handle)
00429 * @note Not thread-safe; call from single initialization context
00430 *
00431 * @see rx_uart_comm_deinit() Cleanup resources
00432 * @see uart_channel_t UART channel enumeration
00433 *
00434 * @since Version 1.0.0
00435 */
00436 [[nodiscard]] rx_err_t rx_uart_comm_init(rx_uart_comm_handle_t* handle,
00437                                         const rx_uart_comm_config_t* config);
00438
00439 /**
00440 * @brief Deinitialize UART communication handle
00441 *
00442 * @details
00443 * Clears the initialized flag to prevent further use of the handle.
00444 * Does not deinitialize the underlying UART hardware.
00445 *
00446 * @param[in,out] handle Pointer to initialized handle
00447 * @return k_rx_ok on success, k_rx_err_invalid_arg if handle is nullptr
00448 *
00449 * @pre handle must not be nullptr
00450 * @pre handle must have been initialized via rx_uart_comm_init()
00451 * @post handle->initialized == 0
00452 * @post Further calls to rx_uart_comm_send/receive will return k_rx_err_invalid_state
00453 *
00454 * @since Version 1.0.0
00455 */
00456 [[nodiscard]] rx_err_t rx_uart_comm_deinit(rx_uart_comm_handle_t* handle);

```

```

00457
00458 /*
=====
00459 * Send API
00460 *
=====
00461 */
00462
00463 /**
00464 * @brief Send a frame over UART (SCI9)
00465 *
00466 * @details
00467 * Encodes the payload into a frame with automatic sequence number assignment
00468 * and CRC-32 calculation, then transmits it via uart_write_channel().
00469 * This operation is non-blocking once the data is queued in the UART FIFO.
00470 *
00471 * @par Algorithm
00472 * 1. Validate handle and payload pointers
00473 * 2. Check initialization
00474 * 3. Check payload fits in TX buffer
00475 * 4. Encode frame with header, payload, and CRC-32
00476 * 5. Write encoded frame to UART via uart_write_channel()
00477 * 6. Increment session TX sequence counter
00478 *
00479 * @param[in,out] handle Initialized handle
00480 * - Must be initialized via rx_uart_comm_init()
00481 * - Session TX sequence incremented on success
00482 * @param[in] type Frame type (typically k_frame_type_response)
00483 * - See rx_frame_type_t for valid types
00484 * @param[in] flags Frame flags (ORed together)
00485 * - See rx_frame_flags_t for valid flags; 0 for no flags
00486 * @param[in] payload Pointer to payload data
00487 * - Must not be nullptr (even for zero-length payload)
00488 * - Copied to internal buffer (no ownership transfer)
00489 * @param[in] payload_len Payload length in bytes
00490 * - Max: k_uart_comm_tx_buffer_size - frame overhead
00491 * - 0 is valid (empty payload)
00492 *
00493 * @return rx_err_t Error code
00494 * @retval k_rx_ok Frame sent successfully
00495 * @retval k_rx_err_invalid_arg handle or payload is nullptr
00496 * @retval k_rx_err_invalid_state Not initialized
00497 * @retval k_rx_err_invalid_size Payload exceeds maximum frame size
00498 * @retval k_rx_err_hw_error UART hardware write failure (propagated from uart_write_channel())
00499 * @retval other Propagated from rx_session_next_tx() or rx_frame_encode()
00500 *
00501 * @pre handle initialized via rx_uart_comm_init()
00502 * @pre payload != nullptr
00503 * @post On success: session TX sequence incremented
00504 * @post On success: frame transmitted via UART
00505 * @post On failure: no state changes
00506 *
00507 * @note Thread safety: caller must provide external synchronization
00508 * @warning When SCI9 is used for comm, debug log output to SCI9 is suppressed
00509 *
00510 * @see rx_uart_comm_receive() Receive frames from peer
00511 *
00512 * @since Version 1.0.0
00513 */
00514 [[nodiscard]] rx_err_t rx_uart_comm_send(rx_uart_comm_handle_t* handle,
00515                                         rx_frame_type_t      type,
00516                                         uint8_t             flags,
00517                                         const uint8_t*        payload,
00518                                         uint32_t             payload_len);
00519
00520 /*
=====
00521 * Receive API
00522 *
=====
00523 */
00524
00525 /**
00526 * @brief Receive and decode a frame from UART (SCI9)
00527 *
00528 * @details
00529 * Reads available bytes from the UART RX buffer, feeds them through the
00530 * frame decoder, validates CRC-32, and extracts the payload. Operates in
00531 * polling mode (non-blocking); returns k_rx_err_no_data if no complete
00532 * frame is available yet.
00533 *
00534 * @par Algorithm
00535 * 1. Validate handle and frame pointers
00536 * 2. Check initialization
00537 * 3. Check uart_rx_available() for pending data
00538 * 4. If data available: read bytes via uart_read_channel()
00539 * 5. Feed bytes to frame decoder

```

```

00540 * 6. If frame complete: validate CRC, populate output frame
00541 * 7. Update session RX sequence
00542 *
00543 * @param[in,out] handle Initialized handle
00544 * - Must be initialized via rx_uart_comm_init()
00545 * - Internal buffers and decoder state modified on each call
00546 * @param[out] frame Pointer to frame structure for output
00547 * - Must be valid non-nullptr
00548 * - All fields populated on success
00549 * @param[in] timeout_ms Timeout in milliseconds
00550 * - 0: Poll once, return immediately if no complete frame
00551 * - (timeout not currently used; always polls once)
00552 *
00553 * @return rx_err_t Error code
00554 * @retval k_rx_ok Complete frame received and decoded
00555 * @retval k_rx_err_invalid_arg handle or frame is nullptr
00556 * @retval k_rx_err_invalid_state Not initialized
00557 * @retval k_rx_err_no_data No complete frame available yet
00558 * @retval k_rx_err_crc_mismatch CRC-32 validation failed
00559 * @retval k_rx_err_protocol_error Invalid frame format
00560 *
00561 * @pre handle initialized via rx_uart_comm_init()
00562 * @pre frame points to valid rx_frame_t
00563 * @post On success: frame contains decoded data
00564 * @post On success: session RX sequence updated
00565 *
00566 * @note Loop iterations bounded by k_uart_comm_max_receive_iterations
00567 * @note Thread safety: caller must provide external synchronization
00568 *
00569 * @see rx_uart_comm_send() Send frames to peer
00570 *
00571 * @since Version 1.0.0
00572 */
00573 [[nodiscard]] rx_err_t
00574 rx_uart_comm_receive(rx_uart_comm_handle_t* handle, rx_frame_t* frame, uint32_t timeout_ms);
00575
00576 #ifdef __cplusplus
00577 }
00578 #endif

```

4.3 libs/rx_uart_comm/src/rx_uart_comm.c File Reference

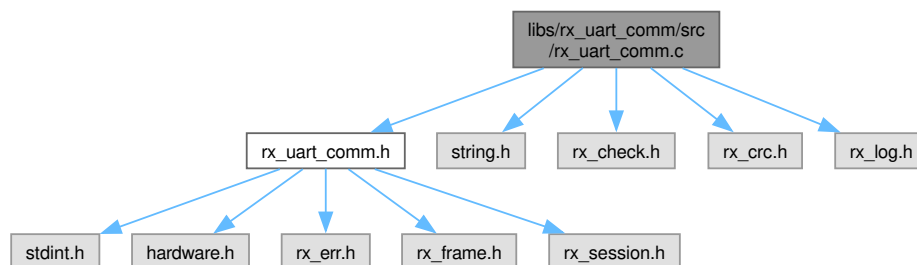
High-Level UART (SCI9) Communication Layer Implementation.

```

#include "rx_uart_comm.h"
#include <string.h>
#include "rx_check.h"
#include "rx_crc.h"
#include "rx_log.h"

```

Include dependency graph for rx_uart_comm.c:



Enumerations

- enum `rx_uart_comm_header_size_t` : `uint8_t { k_frame_header_total }`
Total encoded frame header size in bytes.
- enum `uart_crc32_seed_t` : `uint32_t { k_uart_crc32_seed_initial = 0U }`

- Initial seed value for CRC-32 output variable.
- enum `rx_uart_comm_sequence_constants_t` : `uint16_t` { `k_initial_sequence` = 0 }
Sequence number initial value constants.
- enum `rx_uart_comm_sync_constants_t` : `uint8_t` { `k_sync_second_byte_offset` = 1 }
Sync word search helper constants.
- enum `uart_frame_header_offset_t` : `uint8_t` { `k_hdr_len_offset` = 4 }
Byte offsets into the raw frame header buffer.
- enum `rx_uart_comm_sync_result_t` : `int32_t` { `k_sync_not_found` = -1 }
Sentinel value returned by `internal_find_sync()` when no sync is found.
- enum `rx_receive_result_t` : `uint8_t` { `k_receive_continue` = 0 , `k_receive_done` = 1 , `k_receive_error` = 2 }
Return codes for `internal_receive_iteration()`.
- enum `uart_dispatch_result_t` : `uint8_t` { `k_dispatch_continue` = 0 , `k_dispatch_done` = 1 , `k_dispatch_error` = 2 }
Return codes for `internal_dispatch_frame()`.

Functions

- `RX_STATIC_TESTABLE rx_err_t internal_decode_header` (const `uint8_t` *data, const `uint32_t` data_len, `rx_frame_t` *frame, `uint32_t` *offset_out)
Decode frame header from raw wire data.
- `RX_STATIC_TESTABLE rx_err_t internal_verify_crc` (const `uint8_t` *data, `uint32_t` offset, `uint32_t` *crc_out)
Verify CRC-32 IEEE checksum for received frame.
- `RX_STATIC_TESTABLE rx_err_t internal_read_uart_data` (`rx_uart_comm_handle_t` *handle)
Read available data from UART into staging buffer.
- `RX_STATIC_TESTABLE rx_err_t internal_compact_rx_buffer` (`rx_uart_comm_handle_t` *handle)
Compact receive buffer by removing consumed data.
- `RX_STATIC_TESTABLE rx_err_t internal_find_sync` (const `rx_uart_comm_handle_t` *handle, `int32_t` *sync_pos)
Search for frame sync word in receive buffer.
- `RX_STATIC_TESTABLE rx_err_t internal_handle_no_sync` (`rx_uart_comm_handle_t` *handle)
Handle case when no sync word is found in buffer.
- `RX_STATIC_TESTABLE rx_err_t internal_decode_frame` (`rx_uart_comm_handle_t` *handle, `rx_frame_t` *frame, const `uint32_t` total_size)
Decode a complete frame from buffer.
- `RX_STATIC_TESTABLE void internal_align_to_sync` (`rx_uart_comm_handle_t` *handle, const `int32_t` sync_pos)
Align buffer position to sync word if found.
- `RX_STATIC_TESTABLE rx_err_t internal_parse_header` (`rx_uart_comm_handle_t` *handle, `uint16_t` *payload_len, `uint32_t` *total_size)
Parse frame header and validate payload length.
- `RX_STATIC_TESTABLE rx_receive_result_t internal_receive_iteration` (`rx_uart_comm_handle_t` *handle, `rx_frame_t` *frame, `rx_err_t` *err)
Process one iteration of the frame receive loop.
- `rx_err_t rx_uart_comm_init` (`rx_uart_comm_handle_t` *handle, const `rx_uart_comm_config_t` *config)
Initialize UART communication handle.
- `rx_err_t rx_uart_comm_deinit` (`rx_uart_comm_handle_t` *handle)

Deinitialize UART communication handle.

- RX_STATIC_TESTABLE rx_err_t [internal_build_frame](#) (rx_frame_t *frame, const uint16_t sequence, const rx_frame_type_t type, const uint8_t flags, const uint8_t *payload, const uint32_t payload_len)

Build frame structure from individual parameters.

- rx_err_t [rx_uart_comm_send](#) (rx_uart_comm_handle_t *handle, rx_frame_type_t type, uint8_t flags, const uint8_t *payload, uint32_t payload_len)

Send a frame over UART (SCI9).

- RX_STATIC_TESTABLE rx_err_t [internal_handle_ping](#) (rx_uart_comm_handle_t *handle, const rx_frame_t *frame)

Handle a received PING frame by sending a PONG response.

- RX_STATIC_TESTABLE rx_err_t [internal_handle_reset](#) (rx_uart_comm_handle_t *handle)

Handle a received RESET frame by sending RESET_ACK and resetting session.

- static [uart_dispatch_result_t](#) [internal_dispatch_frame](#) (rx_uart_comm_handle_t *handle, rx_frame_t *frame, rx_err_t *err)

Dispatch a decoded frame by type (PING/RESET/PONG/data).

- rx_err_t [rx_uart_comm_receive](#) (rx_uart_comm_handle_t *handle, rx_frame_t *frame, uint32_t timeout_ms)

Receive and decode a frame from UART (SCI9).

Variables

- static const char *const [s_tag](#) = "UART_COMM"
- static const [rx_uart_comm_handle_t](#) [s_zero_handle](#) = {}

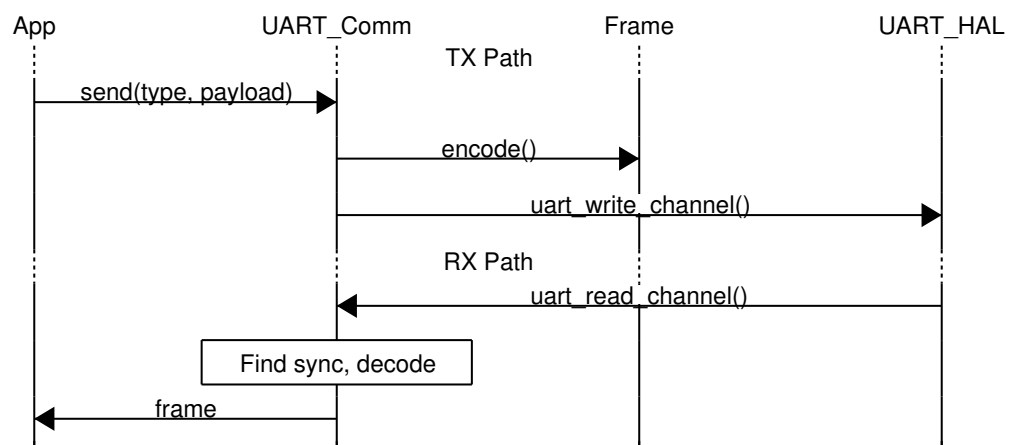
Zero-initialized UART comm handle template for stack-safe initialization.

4.3.1 Detailed Description

High-Level UART (SCI9) Communication Layer Implementation.

Implements reliable UART communication by integrating the frame protocol layer (rx_frame) with the SCI9 UART driver (hardware.h HAL). Handles frame encoding/decoding, CRC-32 validation, sequence number management, and receive buffer handling.

Implementation Architecture:



Channel Assignment:

Channel	Constant	Purpose
SCI9	k_uart_channel_9	Binary protocol

Performance:

Operation	Typical	Worst Case
Send (64B)	6 ms	20 ms
Receive	6 ms	non-blocking

NASA Power of 10 Compliance:

- Rule 1: [OK] No goto, setjmp, recursion
- Rule 2: [OK] Loops bounded by k_uart_comm_max_receive_iterations
- Rule 3: [OK] No dynamic allocation
- Rule 5: [OK] Pre/post conditions on all functions
- Rule 7: [OK] All return values checked
- Rule 8: [OK] Typed enums for constants

SOLID Principles:

- SRP: Handles UART framing only, delegates I/O to UART HAL
- DIP: Uses hardware.h HAL abstraction

See also

[rx_uart_comm.h](#) Public API header
[rx_frame.h](#) Frame encoding layer
[hardware.h](#) UART HAL

Author

Locked, Inc.

Date

2026-03-11

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_uart_comm.c](#).

4.3.2 Enumeration Type Documentation

4.3.2.1 rx_receive_result_t

enum [rx_receive_result_t](#) : uint8_t

Return codes for [internal_receive_iteration\(\)](#).

Allows the receive loop in [rx_uart_comm_receive\(\)](#) to distinguish between "need more data", "frame ready", and "error occurred" without overloading the rx_err_t return type.

Enumerator

k_receive_continue	Continue to next iteration
k_receive_done	Frame received successfully
k_receive_error	Error occurred, check err output

Definition at line 278 of file [rx_uart_comm.c](#).

```
00278      : uint8_t {
00279  k_receive_continue = 0, /**< Continue to next iteration */
00280  k_receive_done     = 1, /**< Frame received successfully */
00281  k_receive_error    = 2, /**< Error occurred, check err output */
00282 } rx_receive_result_t;
```

4.3.2.2 rx_uart_comm_header_size_t

enum [rx_uart_comm_header_size_t](#) : uint8_t

Total encoded frame header size in bytes.

Computed at compile time from the individual field size constants defined in rx_frame.h. Used when determining how many bytes are needed to hold a complete frame header before the payload begins.

Enumerator

k_frame_header_total	Total header bytes
----------------------	--------------------

Definition at line 87 of file [rx_uart_comm.c](#).

```
00087      : uint8_t {
00088  k_frame_header_total = k_frame_sync_size + k_frame_seq_size + k_frame_len_size +
00089                        k_frame_type_size + k_frame_flags_size, /**< Total header bytes */
00090 } rx_uart_comm_header_size_t;
```

4.3.2.3 rx_uart_comm_sequence_constants_t

enum [rx_uart_comm_sequence_constants_t](#) : uint16_t

Sequence number initial value constants.

Provides a named constant for the sequence number starting value used when initializing a TX sequence or building a reset-ack frame, avoiding magic 0.

Enumerator

k_initial_sequence	Initial TX/RX sequence number
--------------------	-------------------------------

Definition at line 224 of file [rx_uart_comm.c](#).

```
00224      : uint16_t {
00225  k_initial_sequence = 0, /**< Initial TX/RX sequence number */
00226 } rx_uart_comm_sequence_constants_t;
```

4.3.2.4 rx_uart_comm_sync_constants_t

```
enum rx_uart_comm_sync_constants_t : uint8_t
```

Sync word search helper constants.

Provides a named offset for the second byte of the two-byte sync word when scanning the receive buffer for frame boundaries.

Enumerator

k_sync_second_byte_offset	Offset from first sync byte to second sync byte
---------------------------	---

Definition at line 236 of file [rx_uart_comm.c](#).

```
00236      : uint8_t {
00237  k_sync_second_byte_offset = 1, /**< Offset from first sync byte to second sync byte */
00238 } rx_uart_comm_sync_constants_t;
```

4.3.2.5 rx_uart_comm_sync_result_t

```
enum rx_uart_comm_sync_result_t : int32_t
```

Sentinel value returned by [internal_find_sync\(\)](#) when no sync is found.

Uses a signed int32_t so -1 is representable, distinguishing "not found" from any valid non-negative buffer position.

Enumerator

k_sync_not_found	Sync word not found in buffer
------------------	-------------------------------

Definition at line 265 of file [rx_uart_comm.c](#).

```
00265      : int32_t {
00266  k_sync_not_found = -1, /**< Sync word not found in buffer */
00267 } rx_uart_comm_sync_result_t;
```

4.3.2.6 uart_crc32_seed_t

enum [uart_crc32_seed_t](#) : uint32_t

Initial seed value for CRC-32 output variable.

Used to zero-initialize the CRC output variable passed to [rx_crc32_ieee\(\)](#) before the calculation begins, avoiding use of a magic literal 0U.

Enumerator

k_uart_crc32_seed_initial	Initial value for CRC-32 output variable
---------------------------	--

Definition at line 100 of file [rx_uart_comm.c](#).

```
00100      : uint32_t {
00101  k_uart_crc32_seed_initial = 0U, /**< Initial value for CRC-32 output variable */
00102 } uart_crc32_seed_t;
```

4.3.2.7 uart_dispatch_result_t

enum [uart_dispatch_result_t](#) : uint8_t

Return codes for [internal_dispatch_frame\(\)](#).

Distinguishes between "frame consumed internally, continue loop", "frame dispatched to caller as data", and "error during dispatch".

Enumerator

k_dispatch_continue	Control frame consumed; continue receive loop
k_dispatch_done	Data frame validated; return to caller
k_dispatch_error	Error during dispatch; check err output

Definition at line 933 of file [rx_uart_comm.c](#).

```
00933      : uint8_t {
00934  k_dispatch_continue = 0, /**< Control frame consumed; continue receive loop */
00935  k_dispatch_done     = 1, /**< Data frame validated; return to caller */
00936  k_dispatch_error    = 2, /**< Error during dispatch; check err output */
00937 } uart_dispatch_result_t;
```

4.3.2.8 uart_frame_header_offset_t

enum [uart_frame_header_offset_t](#) : uint8_t

Byte offsets into the raw frame header buffer.

Provides named constants for the fixed field positions within the encoded frame header so that [internal_parse_header\(\)](#) avoids magic number indexing.

Enumerator

k_hdr_len_offset	Payload length field byte offset within header
------------------	--

Definition at line 248 of file [rx_uart_comm.c](#).

```
00248      : uint8_t {
00249  k_hdr_len_offset = 4, /**< Payload length field byte offset within header */
00250 } uart_frame_header_offset_t;
```

4.3.3 Function Documentation

4.3.3.1 `internal_align_to_sync()`

```
RX_STATIC_TESTABLE void internal_align_to_sync (
    rx_uart_comm_handle_t * handle,
    const int32_t sync_pos)
```

Align buffer position to sync word if found.

Advances `rx_buffer_pos` to `sync_pos` when `sync_pos` is ahead of the current position, discarding any bytes before the sync word.

Precondition

```
handle != nullptr
sync_pos >= 0
```

Postcondition

```
handle->rx_buffer_pos >= (uint32_t)sync_pos
No bytes before sync_pos remain accessible
```

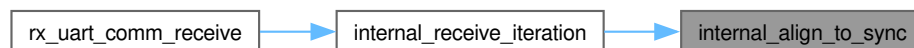
Definition at line 542 of file `rx_uart_comm.c`.

```
00544 {
00545     if ((uint32_t)sync_pos > handle->rx_buffer_pos) {
00546         handle->rx_buffer_pos = (uint32_t)sync_pos;
00547     }
00548 }
```

References `rx_uart_comm_handle_t::rx_buffer_pos`.

Referenced by `internal_receive_iteration()`.

Here is the caller graph for this function:



4.3.3.2 `internal_build_frame()`

```
RX_STATIC_TESTABLE rx_err_t internal_build_frame (
    rx_frame_t * frame,
    const uint16_t sequence,
    const rx_frame_type_t type,
    const uint8_t flags,
    const uint8_t * payload,
    const uint32_t payload_len)
```

Build frame structure from individual parameters.

Populates all header fields of `*frame` and copies the payload bytes into `frame->payload`. Used by `rx_uart_comm_send()` to construct outgoing frames before encoding and transmission.

Precondition

```
frame != nullptr
payload != nullptr || payload_len == 0
```

Postcondition

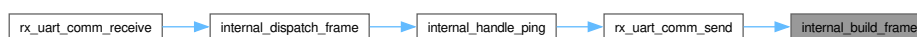
```
frame->header.sequence, type, length, flags are set
frame->payload[0..payload_len-1] copied from payload on k_rx_ok
```

Definition at line 751 of file [rx_uart_comm.c](#).

```
00757 {
00758     if (frame == nullptr) {
00759         return k_rx_err_invalid_arg;
00760     }
00761     if (payload == nullptr && payload_len > 0) {
00762         return k_rx_err_invalid_arg;
00763     }
00764     if (payload_len > k_frame_max_payload) {
00765         return k_rx_err_invalid_size;
00766     }
00767     frame->header.sequence = sequence;
00768     frame->header.length   = (uint16_t)payload_len;
00769     frame->header.type     = (uint8_t)type;
00770     frame->header.flags    = flags;
00771     if (payload != nullptr && payload_len > 0) {
00772         for (uint32_t i = 0; i < payload_len; i++) {
00773             frame->payload[i] = payload[i];
00774         }
00775     }
00776     return k_rx_ok;
00777 }
00778 }
```

Referenced by [rx_uart_comm_send\(\)](#).

Here is the caller graph for this function:



4.3.3.3 internal_compact_rx_buffer()

```
RX_STATIC_TESTABLE rx_err_t internal_compact_rx_buffer (
    rx_uart_comm_handle_t * handle)
```

Compact receive buffer by removing consumed data.

Removes data from the beginning of rx_buffer that has already been processed (bytes 0 to rx_buffer_pos-1). Remaining data is moved to the start of the buffer using memmove() to handle overlapping regions safely.

Precondition

```
handle != nullptr
handle->rx_buffer_pos <= handle->rx_buffer_len
```

Postcondition

handle->rx_buffer_pos == 0

Remaining data preserved at buffer start

Note

Uses memmove() for safe overlapping copy

See also

[internal_read_uart_data\(\)](#) Fills buffer before compaction

Definition at line 364 of file [rx_uart_comm.c](#).

```

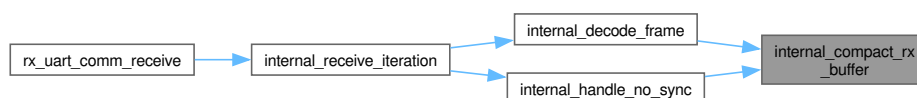
00365 {
00366     if (handle == nullptr) {
00367         return k_rx_err_invalid_arg;
00368     }
00369
00370     if (handle->rx_buffer_pos > handle->rx_buffer_len) {
00371         return k_rx_err_invalid_state;
00372     }
00373
00374     if (handle->rx_buffer_pos == 0) {
00375         return k_rx_ok; /* Nothing to compact */
00376     }
00377
00378     if (handle->rx_buffer_pos >= handle->rx_buffer_len) {
00379         handle->rx_buffer_len = 0;
00380         handle->rx_buffer_pos = 0;
00381     } else {
00382         const uint32_t remaining = handle->rx_buffer_len - handle->rx_buffer_pos;
00383         for (uint32_t i = 0; i < remaining; i++) {
00384             handle->rx_buffer[i] = handle->rx_buffer[handle->rx_buffer_pos + i];
00385         }
00386         handle->rx_buffer_len = remaining;
00387         handle->rx_buffer_pos = 0;
00388     }
00389
00390     /* Post-condition: both branches above set rx_buffer_pos = 0 */
00391
00392     return k_rx_ok;
00393 }

```

References [rx_uart_comm_handle_t::rx_buffer](#), [rx_uart_comm_handle_t::rx_buffer_len](#), and [rx_uart_comm_handle_t::rx_buffer_pos](#).

Referenced by [internal_decode_frame\(\)](#), and [internal_handle_no_sync\(\)](#).

Here is the caller graph for this function:



4.3.3.4 `internal_decode_frame()`

```
RX_STATIC_TESTABLE rx_err_t internal_decode_frame (
    rx_uart_comm_handle_t * handle,
    rx_frame_t * frame,
    const uint32_t total_size)
```

Decode a complete frame from buffer.

Reads `total_size` bytes from the current buffer position, decodes the header, copies the payload, and verifies the CRC-32. Advances `rx_buffer_pos` by `total_size` regardless of success or failure, then compacts the buffer.

Precondition

`handle` != nullptr and initialized
`frame` != nullptr
`total_size` bytes available starting at `handle->rx_buffer_pos`

Postcondition

`rx_buffer_pos` advanced by `total_size`; buffer compacted
`*frame` contains valid decoded frame on `k_rx_ok`

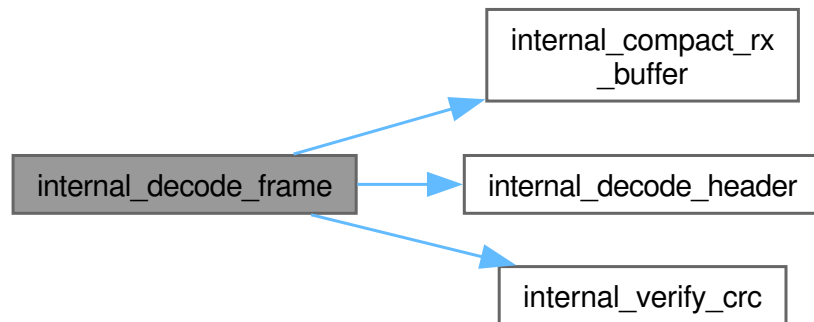
Definition at line 493 of file `rx_uart_comm.c`.

```
00496 {
00497     const uint8_t* hdr    = handle->rx_buffer + handle->rx_buffer_pos;
00498     uint32_t      offset = 0;
00499
00500     (void)internal_decode_header(hdr, total_size, frame, &offset);
00501     /* Post-condition: header decode can only fail if sync word is wrong (impossible: buffer
00502      * was aligned to sync) or payload_len invalid (impossible: parse_header validated it) */
00503
00504     if (frame->header.length > 0) {
00505         for (uint16_t i = 0; i < frame->header.length; i++) {
00506             frame->payload[i] = hdr[offset + i];
00507         }
00508         offset += frame->header.length;
00509     }
00510
00511     rx_err_t err = internal_verify_crc(hdr, offset, &frame->crc);
00512     if (err != k_rx_ok) {
00513         rx_log_error(s_tag, "Frame CRC check failed");
00514     }
00515
00516     handle->rx_buffer_pos += total_size;
00517     (void)internal_compact_rx_buffer(handle);
00518     /* Post-condition: compact only fails on nullptr or pos > len;
00519      * handle is valid and pos <= len is maintained by design */
00520
00521     if (err != k_rx_ok) {
00522         rx_log_error(s_tag, "Frame decode failed");
00523         return err;
00524     }
00525
00526     return k_rx_ok;
00527 }
```

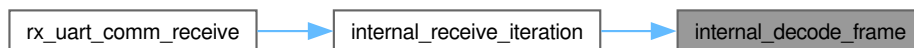
References `internal_compact_rx_buffer()`, `internal_decode_header()`, `internal_verify_crc()`, `rx_uart_comm_handle_t`, `rx_uart_comm_handle_t::rx_buffer_pos`, and `s_tag`.

Referenced by `internal_receive_iteration()`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.5 internal_decode_header()

```

RX_STATIC_TESTABLE rx_err_t internal_decode_header (
    const uint8_t * data,
    const uint32_t data_len,
    rx_frame_t * frame,
    uint32_t * offset_out)
  
```

Decode frame header from raw wire data.

Parses the frame header fields from a raw byte buffer. Validates sync word, extracts sequence number, payload length, type, and flags.

Precondition

```

data != nullptr
frame != nullptr
  
```

Postcondition

```

frame->header fields populated on success
offset_out contains byte offset past header (if not nullptr)
  
```

Note

Not thread-safe, but stateless (safe for concurrent calls with different data)

See also

[internal_verify_crc\(\)](#) Called after this to verify frame CRC

Definition at line 122 of file [rx_uart_comm.c](#).

```

00126 {
00127     if (data == nullptr || frame == nullptr) {
00128         return k_rx_err_invalid_arg;
00129     }
00130
00131     if (data_len < k_frame_min_size) {
00132         return k_rx_err_invalid_size;
00133     }
00134
00135     uint32_t offset = 0;
00136     const uint16_t sync_word = rx_frame_read_le16(&data[offset]);
00137     if (sync_word != k_frame_sync_word) {
00138         return k_rx_err_protocol_error;
00139     }
00140     offset += k_frame_sync_size;
00141
00142     frame->header.sequence = rx_frame_read_le16(&data[offset]);
00143     offset += k_frame_seq_size;
00144
00145     frame->header.length = rx_frame_read_le16(&data[offset]);
00146     offset += k_frame_len_size;
00147
00148     if (frame->header.length > k_frame_max_payload) {
00149         return k_rx_err_invalid_size;
00150     }
00151
00152     (void)(rx_frame_encoded_size(frame->header.length));
00153     /* Post-condition: data_len == total_size which includes the full payload.
00154      * internal_decode_frame is only called after total_size bytes are verified available. */
00155
00156     frame->header.type = data[offset];
00157     offset += k_frame_type_size;
00158
00159     frame->header.flags = data[offset];
00160     offset += k_frame_flags_size;
00161
00162     if (offset_out != nullptr) {
00163         *offset_out = offset;
00164     }
00165
00166     return k_rx_ok;
00167 }

```

Referenced by [internal_decode_frame\(\)](#).

Here is the caller graph for this function:



4.3.3.6 internal_dispatch_frame()

```

uart_dispatch_result_t internal_dispatch_frame (
    rx_uart_comm_handle_t * handle,
    rx_frame_t * frame,
    rx_err_t * err) [static]

```

Dispatch a decoded frame by type (PING/RESET/PONG/data).

Precondition

handle != nullptr and initialized
 frame != nullptr with valid decoded header
 err != nullptr

Postcondition

On `k_dispatch_continue`: PONG/RESET_ACK may have been sent

On `k_dispatch_done`: session RX sequence updated

Definition at line 951 of file `rx_uart_comm.c`.

```

00952 {
00953     /* PING: auto-send PONG, continue loop for next frame */
00954     if (frame->header.type == k_frame_type_ping) {
00955         *err = internal_handle_ping(handle, frame);
00956         return (*err != k_rx_ok) ? k_dispatch_error : k_dispatch_continue;
00957     }
00958
00959     /* RESET: send RESET_ACK, reset session, continue loop */
00960     if (frame->header.type == k_frame_type_reset) {
00961         *err = internal_handle_reset(handle);
00962         return (*err != k_rx_ok) ? k_dispatch_error : k_dispatch_continue;
00963     }
00964
00965     /* PONG / RESET_ACK: consume silently */
00966     if (frame->header.type == k_frame_type_pong || frame->header.type == k_frame_type_reset_ack) {
00967         return k_dispatch_continue;
00968     }
00969
00970     /* Data frame: validate sequence via session */
00971     rx_session_validate_result_t validate_result = k_session_validate_fail;
00972     *err = rx_session_validate_rx(handle->session, frame->header.sequence, &validate_result);
00973     if (*err != k_rx_ok) {
00974         rx_log_error(s_tag, "Session validate_rx returned error");
00975         return k_dispatch_error;
00976     }
00977     return k_dispatch_done;
00978 }

```

References [internal_handle_ping\(\)](#), [internal_handle_reset\(\)](#), [k_dispatch_continue](#), [k_dispatch_done](#), [k_dispatch_error](#), [s_tag](#), and [rx_uart_comm_handle_t::session](#).

Referenced by [rx_uart_comm_receive\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.7 internal_find_sync()

```

RX_STATIC_TESTABLE rx_err_t internal_find_sync (
    const rx_uart_comm_handle_t * handle,
    int32_t * sync_pos)

```

Search for frame sync word in receive buffer.

Scans the receive buffer starting from `rx_buffer_pos` for the frame sync word (0x55AA) stored in little-endian format.

Precondition

```

handle != nullptr
sync_pos != nullptr
handle->rx_buffer_pos <= handle->rx_buffer_len

```

Postcondition

```

sync_pos contains position or k_sync_not_found
Buffer state unchanged

```

Note

O(n) linear scan where n = buffer length

See also

k_frame_sync_word Sync word constant (0x55AA)

Definition at line 414 of file [rx_uart_comm.c](#).

```

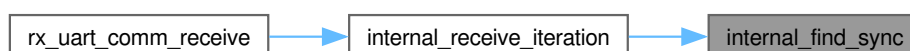
00416 {
00417     if (handle == nullptr) {
00418         return k_rx_err_invalid_arg;
00419     }
00420
00421     if (sync_pos == nullptr) {
00422         return k_rx_err_invalid_arg;
00423     }
00424
00425     if (handle->rx_buffer_pos > handle->rx_buffer_len) {
00426         return k_rx_err_invalid_state;
00427     }
00428
00429     const uint8_t sync_low = (uint8_t)(k_frame_sync_word & k_rx_byte_mask);
00430     const uint8_t sync_high = (uint8_t)(k_frame_sync_word » k_rx_le16_high_shift);
00431
00432     for (uint32_t i = handle->rx_buffer_pos; i + k_sync_second_byte_offset < handle->rx_buffer_len;
00433          i++) {
00434         if (handle->rx_buffer[i] == sync_low &&
00435             handle->rx_buffer[i + k_sync_second_byte_offset] == sync_high) {
00436             *sync_pos = (int32_t)i;
00437             return k_rx_ok;
00438         }
00439     }
00440
00441     *sync_pos = k_sync_not_found;
00442     return k_rx_err_not_found;
00443 }

```

References [k_sync_not_found](#), [k_sync_second_byte_offset](#), [rx_uart_comm_handle_t::rx_buffer](#), [rx_uart_comm_handle_t::rx_buffer_len](#), and [rx_uart_comm_handle_t::rx_buffer_pos](#).

Referenced by [internal_receive_iteration\(\)](#).

Here is the caller graph for this function:



4.3.3.8 `internal_handle_no_sync()`

```
RX_STATIC_TESTABLE rx_err_t internal_handle_no_sync (
    rx_uart_comm_handle_t * handle)
```

Handle case when no sync word is found in buffer.

When no sync word is present, advances the buffer position by one byte if the buffer is full (to make room for new data), then compacts the buffer by discarding consumed bytes from the front.

Precondition

```
handle != nullptr
handle->rx_buffer_pos <= handle->rx_buffer_len
```

Postcondition

```
Buffer compacted; rx_buffer_pos == 0 on success
No sync word present in remaining buffer data
```

Definition at line 460 of file `rx_uart_comm.c`.

```
00461 {
00462     if (handle == nullptr) {
00463         return k_rx_err_invalid_arg;
00464     }
00465
00466     if (handle->rx_buffer_len >= k_uart_comm_rx_buffer_size) {
00467         handle->rx_buffer_pos++;
00468     }
00469
00470     (void)(internal_compact_rx_buffer(handle));
00471     /* Post-condition: compact only fails on nullptr or pos > len; handle is valid
00472      * and pos <= len is maintained as an invariant by all callers */
00473
00474     return k_rx_ok;
00475 }
```

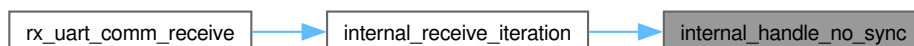
References `internal_compact_rx_buffer()`, `k_uart_comm_rx_buffer_size`, `rx_uart_comm_handle_t::rx_buffer_len`, and `rx_uart_comm_handle_t::rx_buffer_pos`.

Referenced by `internal_receive_iteration()`.

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.9 internal_handle_ping()

```
RX_STATIC_TESTABLE rx_err_t internal_handle_ping (
    rx_uart_comm_handle_t * handle,
    const rx_frame_t * frame)
```

Handle a received PING frame by sending a PONG response.

Sends a PONG frame echoing the received PING payload back over the UART channel. Called automatically by [rx_uart_comm_receive\(\)](#) when a PING frame is decoded.

Precondition

handle != nullptr and handle->initialized == true
 frame != nullptr and frame->header.type == k_frame_type_ping

Postcondition

PONG frame transmitted over UART channel
 Session TX sequence number incremented

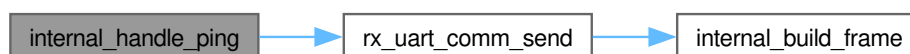
Definition at line 856 of file [rx_uart_comm.c](#).

```
00858 {
00859     if (handle == nullptr || frame == nullptr) {
00860         return k_rx_err_invalid_arg;
00861     }
00862
00863     rx_err_t pong_err = rx_uart_comm_send(handle,
00864                                         k_frame_type_pong,
00865                                         k_frame_flag_none,
00866                                         frame->payload,
00867                                         frame->header.length);
00868     if (pong_err != k_rx_ok) {
00869         rx_log_error(s_tag, "Failed to send PONG response");
00870         return pong_err;
00871     }
00872     rx_log_debug(s_tag, "Auto-responded with PONG");
00873     return k_rx_ok;
00874 }
```

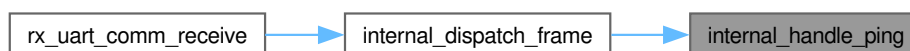
References [rx_uart_comm_send\(\)](#), and [s_tag](#).

Referenced by [internal_dispatch_frame\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.10 `internal_handle_reset()`

```
RX_STATIC_TESTABLE rx_err_t internal_handle_reset (
    rx_uart_comm_handle_t * handle)
```

Handle a received RESET frame by sending RESET_ACK and resetting session.

Constructs and transmits a RESET_ACK frame over the UART channel, then resets the session state machine. Called automatically by `rx_uart_comm_receive()` when a RESET frame is decoded.

Precondition

handle != nullptr and handle->initialized == true
 handle->session != nullptr and initialized

Postcondition

RESET_ACK frame transmitted over UART channel
 handle->session reset to initial state on k_rx_ok

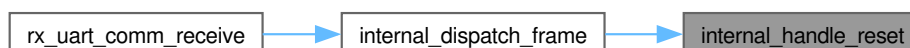
Definition at line 891 of file `rx_uart_comm.c`.

```
00892 {
00893     if (handle == nullptr) {
00894         return k_rx_err_invalid_arg;
00895     }
00896
00897     uint16_t reset_ack_seq = k_initial_sequence;
00898     const rx_err_t seq_err = rx_session_next_tx(handle->session, &reset_ack_seq);
00899     if (seq_err != k_rx_ok) {
00900         return seq_err;
00901     }
00902
00903     rx_frame_t reset_ack_frame = {};
00904     (void)rx_frame_create_reset_ack(&reset_ack_frame, reset_ack_seq);
00905     /* Post-condition: create_reset_ack only fails on nullptr; &reset_ack_frame is never null */
00906
00907     uint32_t wire_len = 0;
00908     (void)rx_frame_encode(&handle->encoder, &reset_ack_frame, handle->tx_buffer, &wire_len);
00909     /* Post-condition: encode only fails on nullptr/uninitialized; handle->encoder is initialized */
00910
00911     rx_err_t write_err = uart_write_channel(handle->channel, handle->tx_buffer, (uint16_t)wire_len);
00912     if (write_err != k_rx_ok) {
00913         rx_log_error(s_tag, "Failed to send RESET_ACK");
00914         return write_err;
00915     }
00916     rx_log_debug(s_tag, "Auto-responded with RESET_ACK");
00917
00918     (void)rx_session_reset(handle->session);
00919     /* Post-condition: session_reset fails only on nullptr or uninitialized session.
00920      * handle->session was validated in rx_uart_comm_init and remains valid. */
00921
00922     return k_rx_ok;
00923 }
```

References `rx_uart_comm_handle_t::channel`, `rx_uart_comm_handle_t::encoder`, `k_initial_sequence`, `s_tag`, `rx_uart_comm_handle_t::session`, and `rx_uart_comm_handle_t::tx_buffer`.

Referenced by `internal_dispatch_frame()`.

Here is the caller graph for this function:



4.3.3.11 `internal_parse_header()`

```
RX_STATIC_TESTABLE rx_err_t internal_parse_header (
    rx_uart_comm_handle_t * handle,
    uint16_t * payload_len,
    uint32_t * total_size)
```

Parse frame header and validate payload length.

Reads the payload length field from the frame header at the current buffer position and validates it against `k_frame_max_payload`. Computes the total expected frame size if the length is valid, or advances the buffer position past the sync word if invalid.

Precondition

`handle` != nullptr with at least `k_frame_header_total` bytes available
`payload_len` != nullptr
`total_size` != nullptr

Postcondition

On `k_rx_ok`: `*total_size` == `k_frame_header_total` + `*payload_len` + `k_frame_crc_size`
 On `k_rx_err_invalid_size`: `rx_buffer_pos` advanced by `k_frame_sync_size`

Definition at line 567 of file `rx_uart_comm.c`.

```
00570 {
00571     const uint8_t* hdr = handle->rx_buffer + handle->rx_buffer_pos;
00572     *payload_len      = rx_frame_read_le16(&hdr[k_hdr_len_offset]);
00573
00574     if (*payload_len > k_frame_max_payload) {
00575         rx_log_warn(s_tag, "Invalid payload length, skipping");
00576         handle->rx_buffer_pos += k_frame_sync_size;
00577         return k_rx_err_invalid_size;
00578     }
00579
00580     *total_size = k_frame_header_total + *payload_len + k_frame_crc_size;
00581     return k_rx_ok;
00582 }
```

References `k_frame_header_total`, `k_hdr_len_offset`, `rx_uart_comm_handle_t::rx_buffer`, `rx_uart_comm_handle_t`, and `s_tag`.

Referenced by `internal_receive_iteration()`.

Here is the caller graph for this function:



4.3.3.12 `internal_read_uart_data()`

```
RX_STATIC_TESTABLE rx_err_t internal_read_uart_data (
    rx_uart_comm_handle_t * handle)
```

Read available data from UART into staging buffer.

Reads any available data from the configured UART channel into the handle's rx_buffer. Appends to existing data without overwriting. Returns immediately if buffer is full or no data available.

Precondition

```
handle != nullptr
handle->rx_buffer_len <= k_uart_comm_rx_buffer_size
```

Postcondition

```
handle->rx_buffer_len <= k_uart_comm_rx_buffer_size
Data appended to handle->rx_buffer[old_len..new_len]
```

Note

Non-blocking: returns immediately with available data

See also

[internal_compact_rx_buffer\(\)](#) Call after consuming data

Definition at line 303 of file `rx_uart_comm.c`.

```
00304 {
00305     if (handle == nullptr) {
00306         return k_rx_err_invalid_arg;
00307     }
00308
00309     /* Post-condition: rx_buffer_len is maintained <= k_uart_comm_rx_buffer_size by design.
00310      * All callers ensure rx_buffer_len starts valid; bytes_read <= space prevents overflow. */
00311
00312     const uint32_t space = k_uart_comm_rx_buffer_size - handle->rx_buffer_len;
00313     if (space == 0) {
00314         return k_rx_ok; /* Buffer full */
00315     }
00316
00317     /* Check if data is available before attempting to read */
00318     bool available = false;
00319     const rx_err_t avail_err = uart_rx_available(handle->channel, &available);
00320     if (avail_err != k_rx_ok) {
00321         return avail_err;
00322     }
00323     if (!available) {
00324         return k_rx_ok; /* No data available */
00325     }
00326
00327     /* Read from UART channel */
00328     uint16_t bytes_read = 0;
00329     const rx_err_t err = uart_read_channel(handle->channel,
00330         handle->rx_buffer + handle->rx_buffer_len,
00331         (uint16_t)space,
00332         &bytes_read);
00333     if (err != k_rx_ok) {
00334         return err;
00335     }
00336
00337     handle->rx_buffer_len += (uint32_t)bytes_read;
00338
00339     /* Post-condition: bytes_read <= space = k_uart_comm_rx_buffer_size - old_len,
00340      * so new rx_buffer_len = old_len + bytes_read <= k_uart_comm_rx_buffer_size */
```

```

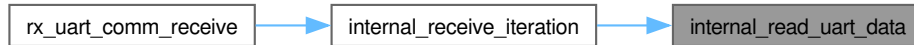
00341
00342  return k_rx_ok;
00343 }

```

References [rx_uart_comm_handle_t::channel](#), [k_uart_comm_rx_buffer_size](#), [rx_uart_comm_handle_t::rx_buffer](#), and [rx_uart_comm_handle_t::rx_buffer_len](#).

Referenced by [internal_receive_iteration\(\)](#).

Here is the caller graph for this function:



4.3.3.13 internal_receive_iteration()

```

RX_STATIC_TESTABLE rx_receive_result_t internal_receive_iteration (
    rx_uart_comm_handle_t * handle,
    rx_frame_t * frame,
    rx_err_t * err)

```

Process one iteration of the frame receive loop.

Performs a single pass of the receive state machine. Attempts to read UART data, find sync word, parse header, and decode complete frame. Returns result code indicating whether to continue, success, or error.

Precondition

handle != nullptr and initialized
 frame != nullptr
 err != nullptr

Postcondition

On k_receive_done: frame contains valid decoded frame
 On k_receive_error: err contains error code

Note

Called in loop by [rx_uart_comm_receive\(\)](#)

See also

[rx_uart_comm_receive\(\)](#) Main receive API that calls this

Definition at line 604 of file [rx_uart_comm.c](#).

```

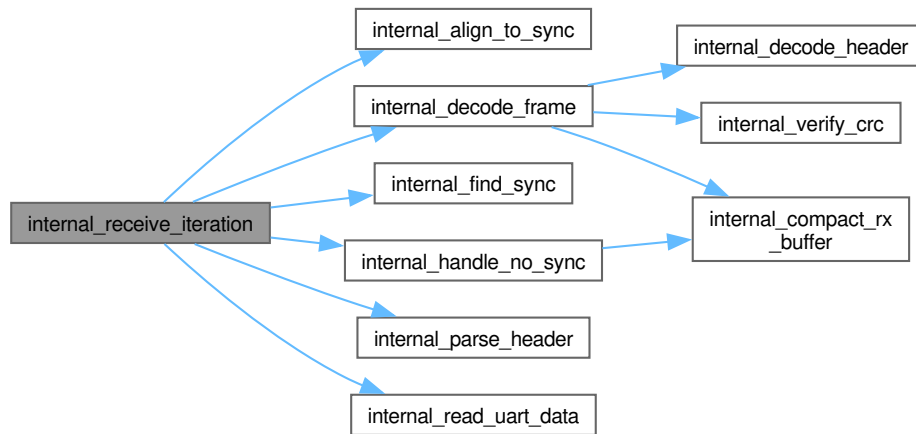
00607 {
00608     /* Read any available UART data */
00609     *err = internal_read_uart_data(handle);
00610     if (*err != k_rx_ok && *err != k_rx_err_timeout) {
00611         return k_receive_error;
00612     }
00613
00614     /* If no data yet, return no_data */
00615     if (handle->rx_buffer_len == handle->rx_buffer_pos) {
00616         *err = k_rx_err_no_data;
00617         return k_receive_error;
00618     }
00619
00620     /* Search for sync word */
00621     int32_t sync_pos = k_sync_not_found;
00622     *err = internal_find_sync(handle, &sync_pos);
00623     if (*err == k_rx_err_not_found) {
00624         *err = internal_handle_no_sync(handle);
00625         /* Post-condition: handle_no_sync only fails if compact fails; compact only fails on
00626          * nullptr/pos>len -- both impossible for a valid, in-use handle */
00627         *err = k_rx_err_no_data;
00628         return k_receive_error;
00629     }
00630     /* Post-condition: find_sync only returns not_found (handled), ok, or error for null/invalid.
00631      * handle is validated before this call so only not_found or ok are reachable. */
00632
00633     /* Found sync - align to it */
00634     internal_align_to_sync(handle, sync_pos);
00635
00636     /* Check if we have enough data for header */
00637     const uint32_t available = handle->rx_buffer_len - handle->rx_buffer_pos;
00638     if (available < k_frame_header_total) {
00639         *err = k_rx_err_no_data;
00640         return k_receive_error;
00641     }
00642
00643     /* Parse and validate header */
00644     uint32_t total_size = 0;
00645     uint16_t payload_len = 0;
00646     *err = internal_parse_header(handle, &payload_len, &total_size);
00647     if (*err == k_rx_err_invalid_size) {
00648         /* Invalid payload length; buffer advanced, continue to find next sync */
00649         *err = k_rx_ok;
00650         return k_receive_continue;
00651     }
00652     /* Post-condition: parse_header only returns invalid_size (handled above) or ok.
00653      * All other error codes are impossible from a valid, initialized handle. */
00654
00655     /* Check if we have complete frame */
00656     if (available < total_size) {
00657         *err = k_rx_err_no_data;
00658         return k_receive_error;
00659     }
00660
00661     /* Decode the complete frame */
00662     *err = internal_decode_frame(handle, frame, total_size);
00663     return (*err != k_rx_ok) ? k_receive_error : k_receive_done;
00664 }

```

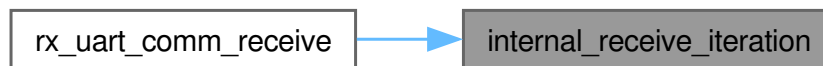
References [internal_align_to_sync\(\)](#), [internal_decode_frame\(\)](#), [internal_find_sync\(\)](#), [internal_handle_no_sync\(\)](#), [internal_parse_header\(\)](#), [internal_read_uart_data\(\)](#), [k_frame_header_total](#), [k_receive_continue](#), [k_receive_done](#), [k_receive_error](#), [k_sync_not_found](#), [rx_uart_comm_handle_t::rx_buffer_len](#), and [rx_uart_comm_handle_t::rx_buffer_pos](#).

Referenced by [rx_uart_comm_receive\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.3.14 internal_verify_crc()

```

RX_STATIC_TESTABLE rx_err_t internal_verify_crc (
    const uint8_t * data,
    uint32_t offset,
    uint32_t * crc_out)

```

Verify CRC-32 IEEE checksum for received frame.

Computes CRC-32 IEEE 802.3 checksum over the frame data (excluding CRC field) and compares against the CRC stored in the frame.

Precondition

```

data != nullptr
offset >= (k_frame_min_size - k_frame_crc_size)

```

Postcondition

```

crc_out contains verified CRC value on success (if not nullptr)
Frame integrity validated on k_rx_ok return

```

Note

Uses `rx_crc32_ieee()` - CRC-32 IEEE 802.3 polynomial

See also

[internal_decode_header\(\)](#) Called before this to parse header

Definition at line 187 of file [rx_uart_comm.c](#).

```

00190 {
00191     if (data == nullptr) {
00192         return k_rx_err_invalid_arg;
00193     }
00194
00195     if (offset < (k_frame_min_size - k_frame_crc_size)) {
00196         return k_rx_err_invalid_arg;
00197     }
00198
00199     const uint32_t received_crc = rx_frame_read_le32(&data[offset]);
00200     uint32_t        calculated_crc = k_uart_crc32_seed_initial;
00201     (void)(rx_crc32_ieee(data, offset, &calculated_crc));
00202     /* Post-condition: rx_crc32_ieee only fails on null/zero-len; data is valid and
00203      * offset >= k_frame_min_size - k_frame_crc_size > 0 is guaranteed by RX_ASSERT above */
00204
00205     if (received_crc != calculated_crc) {
00206         return k_rx_err_crc_mismatch;
00207     }
00208
00209     if (crc_out != nullptr) {
00210         *crc_out = received_crc;
00211     }
00212
00213     return k_rx_ok;
00214 }

```

References [k_uart_crc32_seed_initial](#).

Referenced by [internal_decode_frame\(\)](#).

Here is the caller graph for this function:



4.3.3.15 rx_uart_comm_deinit()

```

rx_err_t rx_uart_comm_deinit (
    rx\_uart\_comm\_handle\_t * handle) [nodiscard]

```

Deinitialize UART communication handle.

Clears the initialized flag to prevent further use of the handle. Does not deinitialize the underlying UART hardware.

Parameters

in,out	handle	Pointer to initialized handle
--------	--------	-------------------------------

Returns

`k_rx_ok` on success, `k_rx_err_invalid_arg` if handle is nullptr

Precondition

handle must not be nullptr

handle must have been initialized via [rx_uart_comm_init\(\)](#)

Postcondition

handle->initialized == 0

Further calls to [rx_uart_comm_send/receive](#) will return [k_rx_err_invalid_state](#)

Since

Version 1.0.0

Definition at line 711 of file [rx_uart_comm.c](#).

```
00712 {
00713     if (handle == nullptr) {
00714         return k_rx_err_invalid_arg;
00715     }
00716     if (handle->initialized) {
00717         (void)(rx_frame_encoder_deinit(&handle->encoder));
00718         /* Post-condition: deinit only fails on nullptr; &handle->encoder is never null */
00719         (void)(rx_frame_decoder_deinit(&handle->decoder));
00720         /* Post-condition: deinit only fails on nullptr; &handle->decoder is never null */
00721         handle->initialized = 0U;
00722     }
00723     rx_log_debug(s_tag, "UART comm deinitialized");
00724     return k_rx_ok;
00725 }
```

References [rx_uart_comm_handle_t::decoder](#), [rx_uart_comm_handle_t::encoder](#), [rx_uart_comm_handle_t::initialized](#) and [s_tag](#).

4.3.3.16 rx_uart_comm_init()

```
rx_err_t rx_uart_comm_init (
    rx_uart_comm_handle_t * handle,
    const rx_uart_comm_config_t * config) [nodiscard]
```

Initialize UART communication handle.

Sets up frame encoder/decoder, clears internal buffers, and prepares the handle for communication. The UART driver must be initialized separately before this handle can successfully send/receive data.

Algorithm

1. Validate handle pointer (NULL check)
2. Clear handle memory
3. Initialize frame encoder and decoder
4. Copy configuration (channel, session)
5. Set initialized flag

Parameters

out	handle	Pointer to handle to initialize • Must be valid non-nullptr • All fields will be overwritten • Caller maintains ownership
in	config	Configuration options (must not be NULL) • session: Required shared session state pointer • channel: UART channel (k_uart_channel_9 for SCI9) • Structure is copied, need not persist after call

Returns

rx_err_t Error code

Return values

k_rx_ok	Handle initialized successfully
k_rx_err_invalid_arg	handle or config is nullptr

Precondition

handle points to allocated [rx_uart_comm_handle_t](#)
 config->session points to initialized rx_session_state_t

Postcondition

handle->initialized == 1
 Buffers cleared to zero

Note

Safe to call multiple times (reinitializes handle)
 Not thread-safe; call from single initialization context

See also

[rx_uart_comm_deinit\(\)](#) Cleanup resources
[uart_channel_t](#) UART channel enumeration

Since

Version 1.0.0

Definition at line 683 of file [rx_uart_comm.c](#).

```

00684 {
00685     if (handle == nullptr) {
00686         return k_rx_err_invalid_arg;
00687     }
00688
00689     *handle = s_zero_handle;
00690
00691     if (config == nullptr || config->session == nullptr) {
00692         return k_rx_err_invalid_arg;
00693     }
00694     handle->session = config->session;
00695     handle->channel = config->channel;
00696
00697     (void)(rx_frame_encoder_init(&handle->encoder));
00698     /* Post-condition: encoder init only fails on nullptr; &handle->encoder is never null */
00699
00700     (void)(rx_frame_decoder_init(&handle->decoder));
00701     /* Post-condition: decoder init only fails on nullptr; &handle->decoder is never null */
00702
00703     handle->rx_buffer_len = 0;
00704     handle->rx_buffer_pos = 0;
00705     handle->initialized = 1U;
00706
00707     rx_log_debug(s_tag, "UART comm initialized");
00708     return k_rx_ok;
00709 }

```

References [rx_uart_comm_config_t::channel](#), [rx_uart_comm_handle_t::channel](#), [rx_uart_comm_handle_t::decoder](#), [rx_uart_comm_handle_t::encoder](#), [rx_uart_comm_handle_t::initialized](#), [rx_uart_comm_handle_t::rx_buffer_len](#), [rx_uart_comm_handle_t::rx_buffer_pos](#), [s_tag](#), [s_zero_handle](#), [rx_uart_comm_config_t::session](#), and [rx_uart_comm_handle_t::session](#).

4.3.3.17 rx_uart_comm_receive()

```

rx_err_t rx_uart_comm_receive (
    rx_uart_comm_handle_t * handle,
    rx_frame_t * frame,
    uint32_t timeout_ms) [nodiscard]

```

Receive and decode a frame from UART (SCI9).

Reads available bytes from the UART RX buffer, feeds them through the frame decoder, validates CRC-32, and extracts the payload. Operates in polling mode (non-blocking); returns `k_rx_err_no_data` if no complete frame is available yet.

Algorithm

1. Validate handle and frame pointers
2. Check initialization
3. Check `uart_rx_available()` for pending data
4. If data available: read bytes via `uart_read_channel()`
5. Feed bytes to frame decoder
6. If frame complete: validate CRC, populate output frame
7. Update session RX sequence

Parameters

in,out	handle	Initialized handle • Must be initialized via rx_uart_comm_init() • Internal buffers and decoder state modified on each call
out	frame	Pointer to frame structure for output • Must be valid non-nullptr • All fields populated on success
in	timeout_ms	Timeout in milliseconds • 0: Poll once, return immediately if no complete frame • (timeout not currently used; always polls once)

Returns

rx_err_t Error code

Return values

k_rx_ok	Complete frame received and decoded
k_rx_err_invalid_arg	handle or frame is nullptr
k_rx_err_invalid_state	Not initialized
k_rx_err_no_data	No complete frame available yet
k_rx_err_crc_mismatch	CRC-32 validation failed
k_rx_err_protocol_error	Invalid frame format

Precondition

handle initialized via [rx_uart_comm_init\(\)](#)

frame points to valid rx_frame_t

Postcondition

On success: frame contains decoded data

On success: session RX sequence updated

Note

Loop iterations bounded by k_uart_comm_max_receive_iterations

Thread safety: caller must provide external synchronization

See also

[rx_uart_comm_send\(\)](#) Send frames to peer

Since

Version 1.0.0

Definition at line 980 of file [rx_uart_comm.c](#).

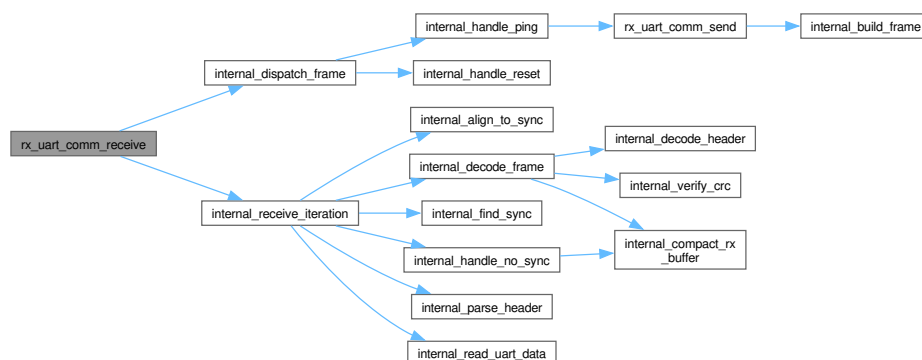
```

00981 {
00982   (void)timeout_ms; /* UART receive is non-blocking; timeout reserved for future use */
00983
00984   if (handle == nullptr) {
00985     return k_rx_err_invalid_arg;
00986   }
00987
00988   if (frame == nullptr) {
00989     return k_rx_err_invalid_arg;
00990   }
00991
00992   if (!handle->initialized) {
00993     return k_rx_err_invalid_state;
00994   }
00995
00996   rx_err_t err = k_rx_ok;
00997
00998   for (uint32_t iterations = 0; iterations < k_uart_comm_max_receive_iterations; iterations++) {
00999     rx_receive_result_t result = internal_receive_iteration(handle, frame, &err);
01000     if (result == k_receive_error) {
01001       return err;
01002     }
01003     if (result != k_receive_done) {
01004       continue;
01005     }
01006
01007     uart_dispatch_result_t dispatch = internal_dispatch_frame(handle, frame, &err);
01008     if (dispatch == k_dispatch_error) {
01009       return err;
01010     }
01011     if (dispatch == k_dispatch_done) {
01012       return k_rx_ok;
01013     }
01014     /* k_dispatch_continue: loop for next frame */
01015   }
01016
01017   return k_rx_err_timeout;
01018 }

```

References [rx_uart_comm_handle_t::initialized](#), [internal_dispatch_frame\(\)](#), [internal_receive_iteration\(\)](#), [k_dispatch_done](#), [k_dispatch_error](#), [k_receive_done](#), [k_receive_error](#), and [k_uart_comm_max_receive_iterations](#).

Here is the call graph for this function:



4.3.3.18 rx_uart_comm_send()

```
rx_err_t rx_uart_comm_send (
    rx_uart_comm_handle_t * handle,
    rx_frame_type_t type,
    uint8_t flags,
    const uint8_t * payload,
    uint32_t payload_len) [nodiscard]
```

Send a frame over UART (SCI9).

Encodes the payload into a frame with automatic sequence number assignment and CRC-32 calculation, then transmits it via `uart_write_channel()`. This operation is non-blocking once the data is queued in the UART FIFO.

Algorithm

1. Validate handle and payload pointers
2. Check initialization
3. Check payload fits in TX buffer
4. Encode frame with header, payload, and CRC-32
5. Write encoded frame to UART via `uart_write_channel()`
6. Increment session TX sequence counter

Parameters

in,out	handle	Initialized handle • Must be initialized via <code>rx_uart_comm_init()</code> • Session TX sequence incremented on success
in	type	Frame type (typically <code>k_frame_type_response</code>) • See <code>rx_frame_type_t</code> for valid types
in	flags	Frame flags (ORed together) • See <code>rx_frame_flags_t</code> for valid flags; 0 for no flags
in	payload	Pointer to payload data • Must not be nullptr (even for zero-length payload) • Copied to internal buffer (no ownership transfer)
in	payload_len	Payload length in bytes • Max: <code>k_uart_comm_tx_buffer_size</code> - frame overhead • 0 is valid (empty payload)

Returns

`rx_err_t` Error code

Return values

<code>k_rx_ok</code>	Frame sent successfully
<code>k_rx_err_invalid_arg</code>	handle or payload is nullptr
<code>k_rx_err_invalid_state</code>	Not initialized
<code>k_rx_err_invalid_size</code>	Payload exceeds maximum frame size
<code>k_rx_err_hw_error</code>	UART hardware write failure (propagated from <code>uart_write_channel()</code>)
other	Propagated from <code>rx_session_next_tx()</code> or <code>rx_frame_encode()</code>

Precondition

handle initialized via [rx_uart_comm_init\(\)](#)
 payload != nullptr

Postcondition

On success: session TX sequence incremented
 On success: frame transmitted via UART
 On failure: no state changes

Note

Thread safety: caller must provide external synchronization

Warning

When SCI9 is used for comm, debug log output to SCI9 is suppressed

See also

[rx_uart_comm_receive\(\)](#) Receive frames from peer

Since

Version 1.0.0

Definition at line 784 of file [rx_uart_comm.c](#).

```

00789 {
00790     if (handle == nullptr) {
00791         return k_rx_err_invalid_arg;
00792     }
00793
00794     if (!handle->initialized) {
00795         rx_log_error(s_tag, "Handle not initialized");
00796         return k_rx_err_invalid_state;
00797     }
00798
00799     if (payload == nullptr && payload_len > 0) {
00800         return k_rx_err_invalid_arg;

```

```

00801 }
00802
00803 if (payload_len > k_frame_max_payload) {
00804     rx_log_error(s_tag, "Payload too large");
00805     return k_rx_err_invalid_size;
00806 }
00807
00808 uint16_t sequence = k_initial_sequence;
00809 rx_err_t err = rx_session_next_tx(handle->session, &sequence);
00810 if (err != k_rx_ok) {
00811     rx_log_error(s_tag, "Session next_tx failed");
00812     return err;
00813 }
00814
00815 rx_frame_t frame = {};
00816 /* build_frame only fails on nullptr frame (impossible: &frame), payload_len > max
00817 * (already validated above), or null payload with nonzero len (validated above) */
00818 (void)internal_build_frame(&frame, sequence, type, flags, payload, payload_len);
00819
00820 uint32_t wire_len = 0;
00821 err = rx_frame_encode(&handle->encoder, &frame, handle->tx_buffer, &wire_len);
00822 if (err != k_rx_ok) {
00823     rx_log_error(s_tag, "Frame encode failed");
00824     return err;
00825 }
00826
00827 err = uart_write_channel(handle->channel, handle->tx_buffer, (uint16_t)wire_len);
00828 if (err != k_rx_ok) {
00829     rx_log_error(s_tag, "UART write failed");
00830     return err;
00831 }
00832
00833 return k_rx_ok;
00834 }

```

References [rx_uart_comm_handle_t::channel](#), [rx_uart_comm_handle_t::encoder](#), [rx_uart_comm_handle_t::initialize](#), [internal_build_frame\(\)](#), [k_initial_sequence](#), [s_tag](#), [rx_uart_comm_handle_t::session](#), and [rx_uart_comm_handle_t::](#)

Referenced by [internal_handle_ping\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



4.3.4 Variable Documentation

4.3.4.1 s_tag

```
const char* const s_tag = "UART_COMM" [static]
```

Definition at line 69 of file [rx_uart_comm.c](#).

Referenced by [internal_decode_frame\(\)](#), [internal_dispatch_frame\(\)](#), [internal_handle_ping\(\)](#), [internal_handle_reset\(\)](#), [internal_parse_header\(\)](#), [rx_uart_comm_deinit\(\)](#), [rx_uart_comm_init\(\)](#), and [rx_uart_comm_send\(\)](#).

4.3.4.2 s_zero_handle

```
const rx_uart_comm_handle_t s_zero_handle = {} [static]
```

Zero-initialized UART comm handle template for stack-safe initialization.

Static const instance used to clear `rx_uart_comm_handle_t` without creating a large compound literal on the stack. Lives in .rodata section.

Note

Read-only; never modified after static initialization

Warning

Do not remove - prevents stack overflow in init function

See also

`rx_uart_comm_init()` Uses this template to zero-initialize the comm handle

Since

Version 1.0.0

Definition at line 681 of file `rx_uart_comm.c`.

```
00681 {};
```

Referenced by `rx_uart_comm_init()`.

4.4 rx_uart_comm.c

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file rx_uart_comm.c
00003  * @brief High-Level UART (SCI9) Communication Layer Implementation
00004  *
00005  * @details
00006  * Implements reliable UART communication by integrating the frame protocol layer
00007  * (rx_frame) with the SCI9 UART driver (hardware.h HAL). Handles frame encoding/decoding,
00008  * CRC-32 validation, sequence number management, and receive buffer handling.
00009  *
00010  * @par Implementation Architecture:
00011  * @msc
00012  * App, UART_Comm, Frame, UART_HAL;
00013  * ... [label="TX Path"];
00014  * App => UART_Comm [label="send(type, payload)"];
00015  * UART_Comm => Frame [label="encode()"];
00016  * UART_Comm => UART_HAL [label="uart_write_channel()"];
00017  * ... [label="RX Path"];
00018  * UART_HAL => UART_Comm [label="uart_read_channel()"];
00019  * UART_Comm box UART_Comm [label="Find sync, decode"];
00020  * UART_Comm => App [label="frame"];
00021  * @endmsc
00022  *
00023  * @par Channel Assignment:
00024  * | Channel | Constant | Purpose |
00025  * |-----|-----|-----|
00026  * | SCI9 | k_uart_channel_9 | Binary protocol |
00027  *
00028  * @par Performance:
```

```

00029 * | Operation      | Typical | Worst Case |
00030 * |-----|-----|-----|
00031 * | Send (64B)      | 6 ms   | 20 ms      |
00032 * | Receive         | 6 ms   | non-blocking|
00033 *
00034 * @par NASA Power of 10 Compliance:
00035 * - Rule 1: [OK] No goto, setjmp, recursion
00036 * - Rule 2: [OK] Loops bounded by k_uart_comm_max_receive_iterations
00037 * - Rule 3: [OK] No dynamic allocation
00038 * - Rule 5: [OK] Pre/post conditions on all functions
00039 * - Rule 7: [OK] All return values checked
00040 * - Rule 8: [OK] Typed enums for constants
00041 *
00042 * @par SOLID Principles:
00043 * - SRP: Handles UART framing only, delegates I/O to UART HAL
00044 * - DIP: Uses hardware.h HAL abstraction
00045 *
00046 * @see rx_uart_comm.h Public API header
00047 * @see rx_frame.h Frame encoding layer
00048 * @see hardware.h UART HAL
00049 *
00050 * @author Locked, Inc.
00051 * @date 2026-03-11
00052 * @copyright Copyright (c) 2026 Locked Inc.
00053 * SPDX-License-Identifier: MIT
00054 */
00055
00056 #include "rx_uart_comm.h"
00057
00058 #include <string.h>
00059
00060 #include "rx_check.h"
00061 #include "rx_crc.h"
00062 #include "rx_log.h"
00063
00064 /*
=====
00065 * Module State
00066 *
=====
00067 */
00068
00069 static const char* const s_tag = "UART_COMM";
00070
00071 /*
=====
00072 * Frame Header Constants
00073 *
00074 * Frame format: [SYNC(2B)][SEQ(2B)][LEN(2B)][TYPE(1B)][FLAGS(1B)]
00075 *
=====
00076 */
00077
00078 /**
00079 * @enum rx_uart_comm_header_size_t
00080 * @brief Total encoded frame header size in bytes
00081 *
00082 * @details
00083 * Computed at compile time from the individual field size constants defined in
00084 * rx_frame.h. Used when determining how many bytes are needed to hold a complete
00085 * frame header before the payload begins.
00086 */
00087 typedef enum : uint8_t {
00088     k_frame_header_total = k_frame_sync_size + k_frame_seq_size + k_frame_len_size +
00089                             k_frame_type_size + k_frame_flags_size, /**< Total header bytes */
00090 } rx_uart_comm_header_size_t;
00091
00092 /**
00093 * @enum uart_crc32_seed_t
00094 * @brief Initial seed value for CRC-32 output variable
00095 *
00096 * @details
00097 * Used to zero-initialize the CRC output variable passed to rx_crc32_ieee()
00098 * before the calculation begins, avoiding use of a magic literal 0U.
00099 */
00100 typedef enum : uint32_t {
00101     k_uart_crc32_seed_initial = 0U, /**< Initial value for CRC-32 output variable */
00102 } uart_crc32_seed_t;
00103
00104 /**
00105 * @brief Decode frame header from raw wire data
00106 *
00107 * @details
00108 * Parses the frame header fields from a raw byte buffer. Validates sync word,
00109 * extracts sequence number, payload length, type, and flags.
00110 *
00111 */

```

```

00112 *
00113 * @pre data != nullptr
00114 * @pre frame != nullptr
00115 * @post frame->header fields populated on success
00116 * @post offset_out contains byte offset past header (if not nullptr)
00117 *
00118 * @note Not thread-safe, but stateless (safe for concurrent calls with different data)
00119 *
00120 * @see internal_verify_crc() Called after this to verify frame CRC
00121 */
00122 RX_STATIC_TESTABLE rx_err_t internal_decode_header(const uint8_t* data,
00123                                                    const uint32_t data_len,
00124                                                    rx_frame_t* frame,
00125                                                    uint32_t* offset_out)
00126 {
00127     if (data == nullptr || frame == nullptr) {
00128         return k_rx_err_invalid_arg;
00129     }
00130
00131     if (data_len < k_frame_min_size) {
00132         return k_rx_err_invalid_size;
00133     }
00134
00135     uint32_t offset = 0;
00136     const uint16_t sync_word = rx_frame_read_le16(&data[offset]);
00137     if (sync_word != k_frame_sync_word) {
00138         return k_rx_err_protocol_error;
00139     }
00140     offset += k_frame_sync_size;
00141
00142     frame->header.sequence = rx_frame_read_le16(&data[offset]);
00143     offset += k_frame_seq_size;
00144
00145     frame->header.length = rx_frame_read_le16(&data[offset]);
00146     offset += k_frame_len_size;
00147
00148     if (frame->header.length > k_frame_max_payload) {
00149         return k_rx_err_invalid_size;
00150     }
00151
00152     (void)(rx_frame_encoded_size(frame->header.length));
00153     /* Post-condition: data_len == total_size which includes the full payload.
00154      * internal_decode_frame is only called after total_size bytes are verified available. */
00155
00156     frame->header.type = data[offset];
00157     offset += k_frame_type_size;
00158
00159     frame->header.flags = data[offset];
00160     offset += k_frame_flags_size;
00161
00162     if (offset_out != nullptr) {
00163         *offset_out = offset;
00164     }
00165
00166     return k_rx_ok;
00167 }
00168
00169 /**
00170 * @brief Verify CRC-32 IEEE checksum for received frame
00171 *
00172 * @details
00173 * Computes CRC-32 IEEE 802.3 checksum over the frame data (excluding CRC field)
00174 * and compares against the CRC stored in the frame.
00175 *
00176 *
00177 *
00178 * @pre data != nullptr
00179 * @pre offset >= (k_frame_min_size - k_frame_crc_size)
00180 * @post crc_out contains verified CRC value on success (if not nullptr)
00181 * @post Frame integrity validated on k_rx_ok return
00182 *
00183 * @note Uses rx_crc32_ieee() - CRC-32 IEEE 802.3 polynomial
00184 *
00185 * @see internal_decode_header() Called before this to parse header
00186 */
00187 RX_STATIC_TESTABLE rx_err_t internal_verify_crc(const uint8_t* data,
00188                                                  uint32_t offset,
00189                                                  uint32_t* crc_out)
00190 {
00191     if (data == nullptr) {
00192         return k_rx_err_invalid_arg;
00193     }
00194
00195     if (offset < (k_frame_min_size - k_frame_crc_size)) {
00196         return k_rx_err_invalid_arg;
00197     }
00198

```

```

00199  const uint32_t received_crc  = rx_frame_read_le32(&data[offset]);
00200  uint32_t      calculated_crc = k_uart_crc32_seed_initial;
00201  (void)(rx_crc32_ieee(data, offset, &calculated_crc));
00202  /* Post-condition: rx_crc32_ieee only fails on null/zero-len; data is valid and
00203   * offset >= k_frame_min_size - k_frame_crc_size > 0 is guaranteed by RX_ASSERT above */
00204
00205  if (received_crc != calculated_crc) {
00206      return k_rx_err_crc_mismatch;
00207  }
00208
00209  if (crc_out != nullptr) {
00210      *crc_out = received_crc;
00211  }
00212
00213  return k_rx_ok;
00214 }
00215
00216 /**
00217  * @enum rx_uart_comm_sequence_constants_t
00218  * @brief Sequence number initial value constants
00219  *
00220  * @details
00221  * Provides a named constant for the sequence number starting value used when
00222  * initializing a TX sequence or building a reset-ack frame, avoiding magic 0.
00223  */
00224 typedef enum : uint16_t {
00225     k_initial_sequence = 0, /**< Initial TX/RX sequence number */
00226 } rx_uart_comm_sequence_constants_t;
00227
00228 /**
00229  * @enum rx_uart_comm_sync_constants_t
00230  * @brief Sync word search helper constants
00231  *
00232  * @details
00233  * Provides a named offset for the second byte of the two-byte sync word when
00234  * scanning the receive buffer for frame boundaries.
00235  */
00236 typedef enum : uint8_t {
00237     k_sync_second_byte_offset = 1, /**< Offset from first sync byte to second sync byte */
00238 } rx_uart_comm_sync_constants_t;
00239
00240 /**
00241  * @enum uart_frame_header_offset_t
00242  * @brief Byte offsets into the raw frame header buffer
00243  *
00244  * @details
00245  * Provides named constants for the fixed field positions within the encoded
00246  * frame header so that internal_parse_header() avoids magic number indexing.
00247  */
00248 typedef enum : uint8_t {
00249     k_hdr_len_offset = 4, /**< Payload length field byte offset within header */
00250 } uart_frame_header_offset_t;
00251
00252 /*
=====
00253  * Internal Helpers
00254  *
=====
00255  */
00256
00257 /**
00258  * @enum rx_uart_comm_sync_result_t
00259  * @brief Sentinel value returned by internal_find_sync() when no sync is found
00260  *
00261  * @details
00262  * Uses a signed int32_t so -1 is representable, distinguishing "not found"
00263  * from any valid non-negative buffer position.
00264  */
00265 typedef enum : int32_t {
00266     k_sync_not_found = -1, /**< Sync word not found in buffer */
00267 } rx_uart_comm_sync_result_t;
00268
00269 /**
00270  * @enum rx_receive_result_t
00271  * @brief Return codes for internal_receive_iteration()
00272  *
00273  * @details
00274  * Allows the receive loop in rx_uart_comm_receive() to distinguish between
00275  * "need more data", "frame ready", and "error occurred" without overloading
00276  * the rx_err_t return type.
00277  */
00278 typedef enum : uint8_t {
00279     k_receive_continue = 0, /**< Continue to next iteration */
00280     k_receive_done     = 1, /**< Frame received successfully */
00281     k_receive_error    = 2, /**< Error occurred, check err output */
00282 } rx_receive_result_t;
00283

```

```

00284 /**
00285  * @brief Read available data from UART into staging buffer
00286  *
00287  * @details
00288  * Reads any available data from the configured UART channel into the handle's
00289  * rx_buffer. Appends to existing data without overwriting. Returns immediately
00290  * if buffer is full or no data available.
00291  *
00292  *
00293  *
00294  * @pre handle != nullptr
00295  * @pre handle->rx_buffer_len <= k_uart_comm_rx_buffer_size
00296  * @post handle->rx_buffer_len <= k_uart_comm_rx_buffer_size
00297  * @post Data appended to handle->rx_buffer[old_len..new_len]
00298  *
00299  * @note Non-blocking: returns immediately with available data
00300  *
00301  * @see internal_compact_rx_buffer() Call after consuming data
00302  */
00303 RX_STATIC_TESTABLE rx_err_t internal_read_uart_data(rx_uart_comm_handle_t* handle)
00304 {
00305     if (handle == nullptr) {
00306         return k_rx_err_invalid_arg;
00307     }
00308
00309     /* Post-condition: rx_buffer_len is maintained <= k_uart_comm_rx_buffer_size by design.
00310      * All callers ensure rx_buffer_len starts valid; bytes_read <= space prevents overflow. */
00311
00312     const uint32_t space = k_uart_comm_rx_buffer_size - handle->rx_buffer_len;
00313     if (space == 0) {
00314         return k_rx_ok; /* Buffer full */
00315     }
00316
00317     /* Check if data is available before attempting to read */
00318     bool available = false;
00319     const rx_err_t avail_err = uart_rx_available(handle->channel, &available);
00320     if (avail_err != k_rx_ok) {
00321         return avail_err;
00322     }
00323     if (!available) {
00324         return k_rx_ok; /* No data available */
00325     }
00326
00327     /* Read from UART channel */
00328     uint16_t bytes_read = 0;
00329     const rx_err_t err = uart_read_channel(handle->channel,
00330                                           handle->rx_buffer + handle->rx_buffer_len,
00331                                           (uint16_t)space,
00332                                           &bytes_read);
00333     if (err != k_rx_ok) {
00334         return err;
00335     }
00336
00337     handle->rx_buffer_len += (uint32_t)bytes_read;
00338
00339     /* Post-condition: bytes_read <= space = k_uart_comm_rx_buffer_size - old_len,
00340      * so new rx_buffer_len = old_len + bytes_read <= k_uart_comm_rx_buffer_size */
00341
00342     return k_rx_ok;
00343 }
00344
00345 /**
00346  * @brief Compact receive buffer by removing consumed data
00347  *
00348  * @details
00349  * Removes data from the beginning of rx_buffer that has already been processed
00350  * (bytes 0 to rx_buffer_pos-1). Remaining data is moved to the start of the
00351  * buffer using memmove() to handle overlapping regions safely.
00352  *
00353  *
00354  *
00355  * @pre handle != nullptr
00356  * @pre handle->rx_buffer_pos <= handle->rx_buffer_len
00357  * @post handle->rx_buffer_pos == 0
00358  * @post Remaining data preserved at buffer start
00359  *
00360  * @note Uses memmove() for safe overlapping copy
00361  *
00362  * @see internal_read_uart_data() Fills buffer before compaction
00363  */
00364 RX_STATIC_TESTABLE rx_err_t internal_compact_rx_buffer(rx_uart_comm_handle_t* handle)
00365 {
00366     if (handle == nullptr) {
00367         return k_rx_err_invalid_arg;
00368     }
00369
00370     if (handle->rx_buffer_pos > handle->rx_buffer_len) {

```

```

00371     return k_rx_err_invalid_state;
00372 }
00373
00374 if (handle->rx_buffer_pos == 0) {
00375     return k_rx_ok; /* Nothing to compact */
00376 }
00377
00378 if (handle->rx_buffer_pos >= handle->rx_buffer_len) {
00379     handle->rx_buffer_len = 0;
00380     handle->rx_buffer_pos = 0;
00381 } else {
00382     const uint32_t remaining = handle->rx_buffer_len - handle->rx_buffer_pos;
00383     for (uint32_t i = 0; i < remaining; i++) {
00384         handle->rx_buffer[i] = handle->rx_buffer[handle->rx_buffer_pos + i];
00385     }
00386     handle->rx_buffer_len = remaining;
00387     handle->rx_buffer_pos = 0;
00388 }
00389
00390 /* Post-condition: both branches above set rx_buffer_pos = 0 */
00391
00392 return k_rx_ok;
00393 }
00394
00395 /**
00396  * @brief Search for frame sync word in receive buffer
00397  *
00398  * @details
00399  * Scans the receive buffer starting from rx_buffer_pos for the frame sync
00400  * word (0x55AA) stored in little-endian format.
00401  *
00402  *
00403  *
00404  * @pre handle != nullptr
00405  * @pre sync_pos != nullptr
00406  * @pre handle->rx_buffer_pos <= handle->rx_buffer_len
00407  * @post sync_pos contains position or k_sync_not_found
00408  * @post Buffer state unchanged
00409  *
00410  * @note O(n) linear scan where n = buffer length
00411  *
00412  * @see k_frame_sync_word Sync word constant (0x55AA)
00413  */
00414 RX_STATIC_TESTABLE rx_err_t internal_find_sync(const rx_uart_comm_handle_t* handle,
00415                                                int32_t* sync_pos)
00416 {
00417     if (handle == nullptr) {
00418         return k_rx_err_invalid_arg;
00419     }
00420
00421     if (sync_pos == nullptr) {
00422         return k_rx_err_invalid_arg;
00423     }
00424
00425     if (handle->rx_buffer_pos > handle->rx_buffer_len) {
00426         return k_rx_err_invalid_state;
00427     }
00428
00429     const uint8_t sync_low = (uint8_t)(k_frame_sync_word & k_rx_byte_mask);
00430     const uint8_t sync_high = (uint8_t)(k_frame_sync_word » k_rx_le16_high_shift);
00431
00432     for (uint32_t i = handle->rx_buffer_pos; i + k_sync_second_byte_offset < handle->rx_buffer_len;
00433          i++) {
00434         if (handle->rx_buffer[i] == sync_low &&
00435             handle->rx_buffer[i + k_sync_second_byte_offset] == sync_high) {
00436             *sync_pos = (int32_t)i;
00437             return k_rx_ok;
00438         }
00439     }
00440
00441     *sync_pos = k_sync_not_found;
00442     return k_rx_err_not_found;
00443 }
00444
00445 /**
00446  * @brief Handle case when no sync word is found in buffer
00447  *
00448  * @details
00449  * When no sync word is present, advances the buffer position by one byte if
00450  * the buffer is full (to make room for new data), then compacts the buffer by
00451  * discarding consumed bytes from the front.
00452  *
00453  *
00454  *
00455  * @pre handle != nullptr
00456  * @pre handle->rx_buffer_pos <= handle->rx_buffer_len
00457  * @post Buffer compacted; rx_buffer_pos == 0 on success

```

```

00458 * @post No sync word present in remaining buffer data
00459 */
00460 RX_STATIC_TESTABLE rx_err_t internal_handle_no_sync(rx_uart_comm_handle_t* handle)
00461 {
00462     if (handle == nullptr) {
00463         return k_rx_err_invalid_arg;
00464     }
00465
00466     if (handle->rx_buffer_len >= k_uart_comm_rx_buffer_size) {
00467         handle->rx_buffer_pos++;
00468     }
00469
00470     (void)(internal_compact_rx_buffer(handle));
00471     /* Post-condition: compact only fails on nullptr or pos > len; handle is valid
00472      * and pos <= len is maintained as an invariant by all callers */
00473     return k_rx_ok;
00474 }
00475
00476 /**
00477  * @brief Decode a complete frame from buffer
00478  *
00479  * @details
00480  * Reads total_size bytes from the current buffer position, decodes the header,
00481  * copies the payload, and verifies the CRC-32. Advances rx_buffer_pos by
00482  * total_size regardless of success or failure, then compacts the buffer.
00483  *
00484  * @pre handle != nullptr and initialized
00485  * @pre frame != nullptr
00486  * @pre total_size bytes available starting at handle->rx_buffer_pos
00487  * @post rx_buffer_pos advanced by total_size; buffer compacted
00488  * @post *frame contains valid decoded frame on k_rx_ok
00489  */
00490 RX_STATIC_TESTABLE rx_err_t internal_decode_frame(rx_uart_comm_handle_t* handle,
00491                                                  rx_frame_t* frame,
00492                                                  const uint32_t total_size)
00493 {
00494     const uint8_t* hdr = handle->rx_buffer + handle->rx_buffer_pos;
00495     uint32_t offset = 0;
00496
00497     (void)internal_decode_header(hdr, total_size, frame, &offset);
00498     /* Post-condition: header decode can only fail if sync word is wrong (impossible: buffer
00499      * was aligned to sync) or payload_len invalid (impossible: parse_header validated it) */
00500
00501     if (frame->header.length > 0) {
00502         for (uint16_t i = 0; i < frame->header.length; i++) {
00503             frame->payload[i] = hdr[offset + i];
00504         }
00505         offset += frame->header.length;
00506     }
00507
00508     rx_err_t err = internal_verify_crc(hdr, offset, &frame->crc);
00509     if (err != k_rx_ok) {
00510         rx_log_error(s_tag, "Frame CRC check failed");
00511     }
00512
00513     handle->rx_buffer_pos += total_size;
00514     (void)(internal_compact_rx_buffer(handle));
00515     /* Post-condition: compact only fails on nullptr or pos > len;
00516      * handle is valid and pos <= len is maintained by design */
00517
00518     if (err != k_rx_ok) {
00519         rx_log_error(s_tag, "Frame decode failed");
00520         return err;
00521     }
00522
00523     return k_rx_ok;
00524 }
00525
00526 /**
00527  * @brief Align buffer position to sync word if found
00528  *
00529  * @details
00530  * Advances rx_buffer_pos to sync_pos when sync_pos is ahead of the current
00531  * position, discarding any bytes before the sync word.
00532  *
00533  * @pre handle != nullptr
00534  * @pre sync_pos >= 0
00535  * @post handle->rx_buffer_pos >= (uint32_t)sync_pos
00536  * @post No bytes before sync_pos remain accessible
00537  */
00538 RX_STATIC_TESTABLE void internal_align_to_sync(rx_uart_comm_handle_t* handle,
00539                                               const int32_t sync_pos)
00540 {

```

```

00545 if((uint32_t)sync_pos > handle->rx_buffer_pos) {
00546     handle->rx_buffer_pos = (uint32_t)sync_pos;
00547 }
00548 }
00549
00550 /**
00551  * @brief Parse frame header and validate payload length
00552  *
00553  * @details
00554  * Reads the payload length field from the frame header at the current buffer
00555  * position and validates it against k_frame_max_payload. Computes the total
00556  * expected frame size if the length is valid, or advances the buffer position
00557  * past the sync word if invalid.
00558  *
00559  *
00560  *
00561  * @pre handle != nullptr with at least k_frame_header_total bytes available
00562  * @pre payload_len != nullptr
00563  * @pre total_size != nullptr
00564  * @post On k_rx_ok: *total_size == k_frame_header_total + *payload_len + k_frame_crc_size
00565  * @post On k_rx_err_invalid_size: rx_buffer_pos advanced by k_frame_sync_size
00566  */
00567 RX_STATIC_TESTABLE rx_err_t internal_parse_header(rx_uart_comm_handle_t* handle,
00568                                                    uint16_t* payload_len,
00569                                                    uint32_t* total_size)
00570 {
00571     const uint8_t* hdr = handle->rx_buffer + handle->rx_buffer_pos;
00572     *payload_len = rx_frame_read_le16(&hdr[k_hdr_len_offset]);
00573
00574     if (*payload_len > k_frame_max_payload) {
00575         rx_log_warn(s_tag, "Invalid payload length, skipping");
00576         handle->rx_buffer_pos += k_frame_sync_size;
00577         return k_rx_err_invalid_size;
00578     }
00579
00580     *total_size = k_frame_header_total + *payload_len + k_frame_crc_size;
00581     return k_rx_ok;
00582 }
00583
00584 /**
00585  * @brief Process one iteration of the frame receive loop
00586  *
00587  * @details
00588  * Performs a single pass of the receive state machine. Attempts to read UART
00589  * data, find sync word, parse header, and decode complete frame. Returns
00590  * result code indicating whether to continue, success, or error.
00591  *
00592  *
00593  *
00594  * @pre handle != nullptr and initialized
00595  * @pre frame != nullptr
00596  * @pre err != nullptr
00597  * @post On k_receive_done: frame contains valid decoded frame
00598  * @post On k_receive_error: err contains error code
00599  *
00600  * @note Called in loop by rx_uart_comm_receive()
00601  *
00602  * @see rx_uart_comm_receive() Main receive API that calls this
00603  */
00604 RX_STATIC_TESTABLE rx_receive_result_t internal_receive_iteration(rx_uart_comm_handle_t* handle,
00605                                                                    rx_frame_t* frame,
00606                                                                    rx_err_t* err)
00607 {
00608     /* Read any available UART data */
00609     *err = internal_read_uart_data(handle);
00610     if (*err != k_rx_ok && *err != k_rx_err_timeout) {
00611         return k_receive_error;
00612     }
00613
00614     /* If no data yet, return no_data */
00615     if (handle->rx_buffer_len == handle->rx_buffer_pos) {
00616         *err = k_rx_err_no_data;
00617         return k_receive_error;
00618     }
00619
00620     /* Search for sync word */
00621     int32_t sync_pos = k_sync_not_found;
00622     *err = internal_find_sync(handle, &sync_pos);
00623     if (*err == k_rx_err_not_found) {
00624         *err = internal_handle_no_sync(handle);
00625         /* Post-condition: handle_no_sync only fails if compact fails; compact only fails on
00626          * nullptr/pos>len -- both impossible for a valid, in-use handle */
00627         *err = k_rx_err_no_data;
00628         return k_receive_error;
00629     }
00630     /* Post-condition: find_sync only returns not_found (handled), ok, or error for null/invalid.
00631      * handle is validated before this call so only not_found or ok are reachable. */

```

```

00632
00633 /* Found sync - align to it */
00634 internal_align_to_sync(handle, sync_pos);
00635
00636 /* Check if we have enough data for header */
00637 const uint32_t available = handle->rx_buffer_len - handle->rx_buffer_pos;
00638 if (available < k_frame_header_total) {
00639     *err = k_rx_err_no_data;
00640     return k_receive_error;
00641 }
00642
00643 /* Parse and validate header */
00644 uint32_t total_size = 0;
00645 uint16_t payload_len = 0;
00646 *err = internal_parse_header(handle, &payload_len, &total_size);
00647 if (*err == k_rx_err_invalid_size) {
00648     /* Invalid payload length; buffer advanced, continue to find next sync */
00649     *err = k_rx_ok;
00650     return k_receive_continue;
00651 }
00652 /* Post-condition: parse_header only returns invalid_size (handled above) or ok.
00653 * All other error codes are impossible from a valid, initialized handle. */
00654
00655 /* Check if we have complete frame */
00656 if (available < total_size) {
00657     *err = k_rx_err_no_data;
00658     return k_receive_error;
00659 }
00660
00661 /* Decode the complete frame */
00662 *err = internal_decode_frame(handle, frame, total_size);
00663 return (*err != k_rx_ok) ? k_receive_error : k_receive_done;
00664 }
00665
00666 /*
=====
00667 * Initialization
00668 *
=====
00669 */
00670
00671 /**
00672 * @var s_zero_handle
00673 * @brief Zero-initialized UART comm handle template for stack-safe initialization
00674 * @details Static const instance used to clear rx_uart_comm_handle_t without creating
00675 *         a large compound literal on the stack. Lives in .rodata section.
00676 * @note Read-only; never modified after static initialization
00677 * @warning Do not remove - prevents stack overflow in init function
00678 * @see rx_uart_comm_init() Uses this template to zero-initialize the comm handle
00679 * @since Version 1.0.0
00680 */
00681 static const rx_uart_comm_handle_t s_zero_handle = {};
00682
00683 rx_err_t rx_uart_comm_init(rx_uart_comm_handle_t* handle, const rx_uart_comm_config_t* config)
00684 {
00685     if (handle == nullptr) {
00686         return k_rx_err_invalid_arg;
00687     }
00688
00689     *handle = s_zero_handle;
00690
00691     if (config == nullptr || config->session == nullptr) {
00692         return k_rx_err_invalid_arg;
00693     }
00694     handle->session = config->session;
00695     handle->channel = config->channel;
00696
00697     (void)(rx_frame_encoder_init(&handle->encoder));
00698     /* Post-condition: encoder init only fails on nullptr; &handle->encoder is never null */
00699
00700     (void)(rx_frame_decoder_init(&handle->decoder));
00701     /* Post-condition: decoder init only fails on nullptr; &handle->decoder is never null */
00702
00703     handle->rx_buffer_len = 0;
00704     handle->rx_buffer_pos = 0;
00705     handle->initialized = 1U;
00706
00707     rx_log_debug(s_tag, "UART comm initialized");
00708     return k_rx_ok;
00709 }
00710
00711 rx_err_t rx_uart_comm_deinit(rx_uart_comm_handle_t* handle)
00712 {
00713     if (handle == nullptr) {
00714         return k_rx_err_invalid_arg;
00715     }
00716

```

```

00717 if (handle->initialized) {
00718     (void)(rx_frame_encoder_deinit(&handle->encoder));
00719     /* Post-condition: deinit only fails on nullptr; &handle->encoder is never null */
00720
00721     (void)(rx_frame_decoder_deinit(&handle->decoder));
00722     /* Post-condition: deinit only fails on nullptr; &handle->decoder is never null */
00723
00724     handle->initialized = 0U;
00725 }
00726
00727 rx_log_debug(s_tag, "UART comm deinitialized");
00728 return k_rx_ok;
00729 }
00730
00731 /*
=====
00732 * Send API
00733 *
=====
00734 */
00735 /**
00736  * @brief Build frame structure from individual parameters
00737  *
00738  * @details
00739  * Populates all header fields of *frame and copies the payload bytes into
00740  * frame->payload. Used by rx_uart_comm_send() to construct outgoing frames
00741  * before encoding and transmission.
00742  *
00743  *
00744  *
00745  *
00746  * @pre frame != nullptr
00747  * @pre payload != nullptr || payload_len == 0
00748  * @post frame->header.sequence, type, length, flags are set
00749  * @post frame->payload[0..payload_len-1] copied from payload on k_rx_ok
00750  */
00751 RX_STATIC_TESTABLE rx_err_t internal_build_frame(rx_frame_t* frame,
00752     const uint16_t sequence,
00753     const rx_frame_type_t type,
00754     const uint8_t flags,
00755     const uint8_t* payload,
00756     const uint32_t payload_len)
00757 {
00758     if (frame == nullptr) {
00759         return k_rx_err_invalid_arg;
00760     }
00761
00762     if (payload == nullptr && payload_len > 0) {
00763         return k_rx_err_invalid_arg;
00764     }
00765
00766     if (payload_len > k_frame_max_payload) {
00767         return k_rx_err_invalid_size;
00768     }
00769
00770     frame->header.sequence = sequence;
00771     frame->header.length = (uint16_t)payload_len;
00772     frame->header.type = (uint8_t)type;
00773     frame->header.flags = flags;
00774
00775     if (payload != nullptr && payload_len > 0) {
00776         for (uint32_t i = 0; i < payload_len; i++) {
00777             frame->payload[i] = payload[i];
00778         }
00779     }
00780
00781     return k_rx_ok;
00782 }
00783
00784 rx_err_t rx_uart_comm_send(rx_uart_comm_handle_t* handle,
00785     rx_frame_type_t type,
00786     uint8_t flags,
00787     const uint8_t* payload,
00788     uint32_t payload_len)
00789 {
00790     if (handle == nullptr) {
00791         return k_rx_err_invalid_arg;
00792     }
00793
00794     if (!handle->initialized) {
00795         rx_log_error(s_tag, "Handle not initialized");
00796         return k_rx_err_invalid_state;
00797     }
00798
00799     if (payload == nullptr && payload_len > 0) {
00800         return k_rx_err_invalid_arg;
00801     }

```

```

00802
00803 if (payload_len > k_frame_max_payload) {
00804     rx_log_error(s_tag, "Payload too large");
00805     return k_rx_err_invalid_size;
00806 }
00807
00808 uint16_t sequence = k_initial_sequence;
00809 rx_err_t err = rx_session_next_tx(handle->session, &sequence);
00810 if (err != k_rx_ok) {
00811     rx_log_error(s_tag, "Session next_tx failed");
00812     return err;
00813 }
00814
00815 rx_frame_t frame = {};
00816 /* build_frame only fails on nullptr frame (impossible: &frame), payload_len > max
00817 * (already validated above), or null payload with nonzero len (validated above) */
00818 (void)internal_build_frame(&frame, sequence, type, flags, payload, payload_len);
00819
00820 uint32_t wire_len = 0;
00821 err = rx_frame_encode(&handle->encoder, &frame, handle->tx_buffer, &wire_len);
00822 if (err != k_rx_ok) {
00823     rx_log_error(s_tag, "Frame encode failed");
00824     return err;
00825 }
00826
00827 err = uart_write_channel(handle->channel, handle->tx_buffer, (uint16_t)wire_len);
00828 if (err != k_rx_ok) {
00829     rx_log_error(s_tag, "UART write failed");
00830     return err;
00831 }
00832
00833 return k_rx_ok;
00834 }
00835
00836 /*
=====
00837 * Receive API
00838 *
=====
00839 */
00840
00841 /**
00842 * @brief Handle a received PING frame by sending a PONG response
00843 *
00844 * @details
00845 * Sends a PONG frame echoing the received PING payload back over the UART
00846 * channel. Called automatically by rx_uart_comm_receive() when a PING frame
00847 * is decoded.
00848 *
00849 *
00850 *
00851 * @pre handle != nullptr and handle->initialized == true
00852 * @pre frame != nullptr and frame->header.type == k_frame_type_ping
00853 * @post PONG frame transmitted over UART channel
00854 * @post Session TX sequence number incremented
00855 */
00856 RX_STATIC_TESTABLE rx_err_t internal_handle_ping(rx_uart_comm_handle_t* handle,
00857     const rx_frame_t* frame)
00858 {
00859     if (handle == nullptr || frame == nullptr) {
00860         return k_rx_err_invalid_arg;
00861     }
00862
00863     rx_err_t pong_err = rx_uart_comm_send(handle,
00864         k_frame_type_pong,
00865         k_frame_flag_none,
00866         frame->payload,
00867         frame->header.length);
00868     if (pong_err != k_rx_ok) {
00869         rx_log_error(s_tag, "Failed to send PONG response");
00870         return pong_err;
00871     }
00872     rx_log_debug(s_tag, "Auto-responded with PONG");
00873     return k_rx_ok;
00874 }
00875
00876 /**
00877 * @brief Handle a received RESET frame by sending RESET_ACK and resetting session
00878 *
00879 * @details
00880 * Constructs and transmits a RESET_ACK frame over the UART channel, then resets
00881 * the session state machine. Called automatically by rx_uart_comm_receive() when
00882 * a RESET frame is decoded.
00883 *
00884 *
00885 *
00886 * @pre handle != nullptr and handle->initialized == true

```

```

00887 * @pre handle->session != nullptr and initialized
00888 * @post RESET_ACK frame transmitted over UART channel
00889 * @post handle->session reset to initial state on k_rx_ok
00890 */
00891 RX_STATIC_TESTABLE rx_err_t internal_handle_reset(rx_uart_comm_handle_t* handle)
00892 {
00893     if (handle == nullptr) {
00894         return k_rx_err_invalid_arg;
00895     }
00896     uint16_t reset_ack_seq = k_initial_sequence;
00897     const rx_err_t seq_err = rx_session_next_tx(handle->session, &reset_ack_seq);
00898     if (seq_err != k_rx_ok) {
00899         return seq_err;
00900     }
00901 }
00902
00903 rx_frame_t reset_ack_frame = {};
00904 (void)(rx_frame_create_reset_ack(&reset_ack_frame, reset_ack_seq));
00905 /* Post-condition: create_reset_ack only fails on nullptr; &reset_ack_frame is never null */
00906
00907 uint32_t wire_len = 0;
00908 (void)(rx_frame_encode(&handle->encoder, &reset_ack_frame, handle->tx_buffer, &wire_len));
00909 /* Post-condition: encode only fails on nullptr/uninitialized; handle->encoder is initialized */
00910
00911 rx_err_t write_err = uart_write_channel(handle->channel, handle->tx_buffer, (uint16_t)wire_len);
00912 if (write_err != k_rx_ok) {
00913     rx_log_error(s_tag, "Failed to send RESET_ACK");
00914     return write_err;
00915 }
00916 rx_log_debug(s_tag, "Auto-responded with RESET_ACK");
00917
00918 (void)(rx_session_reset(handle->session));
00919 /* Post-condition: session_reset fails only on nullptr or uninitialized session.
00920 * handle->session was validated in rx_uart_comm_init and remains valid. */
00921
00922 return k_rx_ok;
00923 }
00924
00925 /**
00926 * @enum uart_dispatch_result_t
00927 * @brief Return codes for internal_dispatch_frame()
00928 *
00929 * @details
00930 * Distinguishes between "frame consumed internally, continue loop",
00931 * "frame dispatched to caller as data", and "error during dispatch".
00932 */
00933 typedef enum : uint8_t {
00934     k_dispatch_continue = 0, /**< Control frame consumed; continue receive loop */
00935     k_dispatch_done = 1, /**< Data frame validated; return to caller */
00936     k_dispatch_error = 2, /**< Error during dispatch; check err output */
00937 } uart_dispatch_result_t;
00938
00939 /**
00940 * @brief Dispatch a decoded frame by type (PING/RESET/PONG/data)
00941 *
00942 *
00943 *
00944 * @pre handle != nullptr and initialized
00945 * @pre frame != nullptr with valid decoded header
00946 * @pre err != nullptr
00947 * @post On k_dispatch_continue: PONG/RESET_ACK may have been sent
00948 * @post On k_dispatch_done: session RX sequence updated
00949 */
00950 static uart_dispatch_result_t
00951 internal_dispatch_frame(rx_uart_comm_handle_t* handle, rx_frame_t* frame, rx_err_t* err)
00952 {
00953     /* PING: auto-send PONG, continue loop for next frame */
00954     if (frame->header.type == k_frame_type_ping) {
00955         *err = internal_handle_ping(handle, frame);
00956         return (*err != k_rx_ok) ? k_dispatch_error : k_dispatch_continue;
00957     }
00958
00959     /* RESET: send RESET_ACK, reset session, continue loop */
00960     if (frame->header.type == k_frame_type_reset) {
00961         *err = internal_handle_reset(handle);
00962         return (*err != k_rx_ok) ? k_dispatch_error : k_dispatch_continue;
00963     }
00964
00965     /* PONG / RESET_ACK: consume silently */
00966     if (frame->header.type == k_frame_type_pong || frame->header.type == k_frame_type_reset_ack) {
00967         return k_dispatch_continue;
00968     }
00969
00970     /* Data frame: validate sequence via session */
00971     rx_session_validate_result_t validate_result = k_session_validate_fail;
00972     *err = rx_session_validate_rx(handle->session, frame->header.sequence, &validate_result);
00973     if (*err != k_rx_ok) {

```

```
00974     rx_log_error(s_tag, "Session validate_rx returned error");
00975     return k_dispatch_error;
00976 }
00977 return k_dispatch_done;
00978 }
00979
00980 rx_err_t rx_uart_comm_receive(rx_uart_comm_handle_t* handle, rx_frame_t* frame, uint32_t timeout_ms)
00981 {
00982     (void)timeout_ms; /* UART receive is non-blocking; timeout reserved for future use */
00983
00984     if (handle == nullptr) {
00985         return k_rx_err_invalid_arg;
00986     }
00987
00988     if (frame == nullptr) {
00989         return k_rx_err_invalid_arg;
00990     }
00991
00992     if (!handle->initialized) {
00993         return k_rx_err_invalid_state;
00994     }
00995
00996     rx_err_t err = k_rx_ok;
00997
00998     for (uint32_t iterations = 0; iterations < k_uart_comm_max_receive_iterations; iterations++) {
00999         rx_receive_result_t result = internal_receive_iteration(handle, frame, &err);
01000         if (result == k_receive_error) {
01001             return err;
01002         }
01003         if (result != k_receive_done) {
01004             continue;
01005         }
01006
01007         uart_dispatch_result_t dispatch = internal_dispatch_frame(handle, frame, &err);
01008         if (dispatch == k_dispatch_error) {
01009             return err;
01010         }
01011         if (dispatch == k_dispatch_done) {
01012             return k_rx_ok;
01013         }
01014         /* k_dispatch_continue: loop for next frame */
01015     }
01016
01017     return k_rx_err_timeout;
01018 }
```

